

Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский физико-технический институт
(национальный исследовательский университет)»

«Исследование методов определения характеристик транспортного средства по
видеоизображениям»
(бакалаврская работа)

Студент:
Клименко Виталий Евгеньевич

Научный руководитель:
Назаров Алексей Николаевич

Москва
2026

Аннотация

Ключевые слова: детекция транспортных средств, распознавание номерных знаков, классификация цвета автомобиля, тип кузова, YOLO26, ResNet18, EasyOCR, transfer learning, видеоаналитика, глубокое обучение.

Цель работы — разработка системы автоматического определения характеристик транспортных средств (номерной знак, цвет, тип кузова) по видеопотоку с камер наблюдения. Задачи: анализ существующих методов детекции и классификации; выбор и обоснование архитектур глубокого обучения; подготовка датасетов; обучение и оценка моделей; интеграция в единый конвейер обработки видео.

Предложена конвейерная архитектура: YOLO26n — детекция ТС и номерных знаков, ResNet18 — классификация цвета (9 классов) и типа кузова (6 классов), EasyOCR — распознавание символов кириллицы.

Результаты: детектор номерных знаков YOLO26n достиг $mAP@0.50 = 0.9669$; классификатор цвета — $Accuracy = 0.936$; классификатор типа кузова — $Accuracy = 0.806$; EasyOCR обеспечивает $CRR = 0.750$ на реальных данных.

Рекомендации: заменить EasyOCR специализированным OCR-модулем для российских номеров, расширить набор классов атрибутов, применить квантизацию и pruning моделей для развёртывания на встроенных платформах (NVIDIA Jetson).

Аннотация	1
Обозначения и сокращения	4
1 Введение	5
1.1 Актуальность задачи	5
1.2 Постановка задачи	5
1.3 Обзор методов детекции транспортных средств	5
1.3.1 Сравнение подходов к детекции	5
1.3.2 Двухэтапные детекторы (Faster R-CNN)	6
1.3.3 Одноэтапные детекторы (YOLO)	6
1.4 Обзор методов автоматического распознавания номерных знаков (ANPR)	6
1.4.1 Сравнение ANPR-подходов	6
1.5 Обзор методов распознавания марки и типа кузова (VMMR)	6
1.6 Обзор методов определения цвета автомобиля	7
1.7 Цель и задачи работы	7
2 Описание выбранных методов и датасетов	9
2.1 Архитектура системы	9
2.2 Теоретические основы глубокого обучения	9
2.2.1 Сверточные нейронные сети	9
2.2.2 Функции активации	10
2.2.3 Pooling и агрегация признаков	10
2.2.4 Пакетная нормализация	11
2.3 Детекция транспортных средств: YOLO26	11
2.3.1 Внутренняя архитектура YOLO26	11
2.3.2 Эволюция семейства YOLO	12
2.3.3 Backbone: CSP-сеть	12
2.3.4 Neck: Path Aggregation Network	12
2.3.5 Anchor-free голова и end-to-end инференс без NMS	13
2.3.6 Функция потерь и оптимизация	13
2.3.7 Параметры инференса	14
2.4 Детекция номерных знаков: дообученный YOLO26n	14
2.4.1 Схема transfer learning	14
2.4.2 Параметры обучения	15
2.5 Оптическое распознавание номеров: EasyOCR	15
2.6 Классификация цвета и типа кузова: ResNet18	15
2.6.1 Архитектура ResNet18	15
2.6.2 Двухфазное дообучение	16
2.6.3 Проблема затухающего градиента в глубоких сетях	17
2.6.4 Теоретическое обоснование residual connections	17
2.6.5 ImageNet-предобучение и стратегии transfer learning	17
2.6.6 Гиперпараметры обучения	18
2.6.7 Аугментация данных	18
2.7 Алгоритмы оптимизации	19
2.7.1 Стохастический градиентный спуск с импульсом	19

2.7.2	Adam	19
2.7.3	Планировщики скорости обучения	19
2.8	Методы регуляризации	20
2.8.1	L2-регуляризация (weight decay)	20
2.8.2	Ранняя остановка	20
2.8.3	Аугментация как регуляризация	20
2.8.4	Mixup-аугментация	21
2.8.5	Test-Time Augmentation (TTA)	22
2.9	Сравниваемые архитектуры: ResNet50 и EfficientNet-B0	22
2.9.1	ResNet50: bottleneck-архитектура	23
2.9.2	EfficientNet-B0: составное масштабирование	23
2.10	Датасеты	23
2.10.1	VCoR — классификация цвета	24
2.10.2	CompCars — классификация типа кузова	24
3	Описание экспериментов	25
3.1	Условия проведения экспериментов	25
3.2	Эксперимент 1: дообучение детектора номерных знаков	25
3.2.1	Протокол и обоснование гиперпараметров	25
3.2.2	Результаты	26
3.2.3	Сравнение с YOLOv8n	26
3.3	Эксперимент 2: классификатор цвета	27
3.3.1	Протокол и обоснование гиперпараметров	27
3.3.2	Результаты	27
3.4	Эксперимент 3: классификатор типа кузова	28
3.4.1	Протокол и обоснование гиперпараметров	28
3.4.2	Регуляризация: Mixup-аугментация	29
3.4.3	Результаты	30
3.4.4	Подэксперимент: сравнение архитектур ResNet18, ResNet50 и EfficientNet-B0	31
	Постановка и обоснование методологии	31
	Результаты	31
3.5	Эксперимент 4: финальный конвейер	32
3.5.1	Тест на одиночном изображении	32
3.5.2	Видеоинференс с трекингом и покадровой агрегацией характеристик	33
3.5.3	Метрики производительности	35
3.5.4	Итоговая таблица результатов	35
3.5.5	Детальный анализ ошибок OCR	35
3.5.6	Анализ узких мест и практические рекомендации	37
4	Заключение	38
	Список использованных источников	39

Обозначения и сокращения

ANPR	Automatic Number Plate Recognition — автоматическое распознавание номерных знаков
BN	Batch Normalization — пакетная нормализация
CNN	Convolutional Neural Network — сверточная нейронная сеть
CRR	Character Recognition Rate — доля верно распознанных символов
DCNN	Deep CNN — глубокая сверточная нейронная сеть
FPN	Feature Pyramid Network — сеть пирамиды признаков
GAP	Global Average Pooling — глобальное усредняющее пулирование
IoU	Intersection over Union — пересечение по объединению
LR	Learning Rate — скорость обучения
mAP	mean Average Precision — средняя точность по классам
OCR	Optical Character Recognition — оптическое распознавание символов
ReLU	Rectified Linear Unit — выпрямленная линейная единица
SGD	Stochastic Gradient Descent — стохастический градиентный спуск
SiLU	Sigmoid Linear Unit (Swish)
ТС	Транспортное средство
VMMR	Vehicle Make and Model Recognition — распознавание марки и модели ТС
YOLO	You Only Look Once — одноэтапный детектор объектов

1 Введение

1.1 Актуальность задачи

В настоящее время системы видеонаблюдения широко применяются для мониторинга дорожного движения, обеспечения безопасности и контроля доступа на объекты. Объём видеоданных, поступающих с камер, постоянно растёт, что делает ручную обработку информации неэффективной и трудоёмкой [1].

Практический интерес представляет автоматическое определение характеристик автомобиля: государственного регистрационного знака, марки, модели и цвета. Эти данные применяются в задачах контроля проезда, поиска транспортных средств и учёта трафика [2]. Однако качество видеозаписей с камер наблюдения часто ограничено низким разрешением, изменениями освещённости, погодными условиями и движением объектов.

1.2 Постановка задачи

Задача анализа транспортных средств по видеоданным представляется как последовательность следующих этапов:

1. детекция автомобиля в кадре;
2. локализация номерного знака внутри области автомобиля;
3. оптическое распознавание символов номера;
4. классификация визуальных атрибутов: цвет и тип кузова.

Формально результат детекции объектов на изображении:

$$D = \{(b_i, c_i, p_i)\}_{i=1}^N, \quad (1.1)$$

где $b_i = (x_i, y_i, w_i, h_i)$ — ограничивающий прямоугольник, c_i — класс объекта, p_i — уверенность детекции.

Качество локализации оценивается через Intersection over Union:

$$\text{IoU} = \frac{S(B_{\text{pred}} \cap B_{\text{gt}})}{S(B_{\text{pred}} \cup B_{\text{gt}})}. \quad (1.2)$$

1.3 Обзор методов детекции транспортных средств

1.3.1 Сравнение подходов к детекции

Эволюция методов детекции объектов прошла несколько этапов. Таблица 1.1 суммирует основные подходы.

Table 1.1 – Сравнение методов детекции объектов

Метод	Принцип	Достоинства	mAP	FPS
HOG+SVM	Ручные признаки, скользящее окно	Прост в реализации, нет GPU	низкий	5–15
Нааг-каскады	Каскад слабых классификаторов	Очень быстрые на CPU	низкий	30+
Faster R-CNN	Двухэтапная сеть, RPN+cls	Высокая точность	высокий	5–15
SSD	Одноэтапная, anchor-based	Баланс скорости и точности	средний	46+
YOLO26n	Одноэтапная, anchor-free, NMS-free	Скорость + точность	высокий	100+

1.3.2 Двухэтапные детекторы (Faster R-CNN)

Detectors семейства Faster R-CNN работают в два этапа: Region Proposal Network генерирует кандидаты, затем классификационная голова уточняет координаты. Подход обеспечивает высокую точность, но требователен к вычислительным ресурсам и не обеспечивает скорость, достаточную для обработки видео в реальном времени.

1.3.3 Одноэтапные детекторы (YOLO)

Семейство YOLO [2] решает задачу детекции за один проход по сети. Anchor-free архитектура, CSP-backbone и PAN-FPN neck обеспечивают работу на GPU со скоростью 100+ FPS при конкурентоспособной точности. В качестве детектора в работе используется YOLO26n [3] — актуальная модель семейства (2026), которая дополнительно реализует end-to-end инференс без NMS и отказ от Distribution Focal Loss, что упрощает экспорт и снижает задержку (см. раздел 2.3).

Обоснование выбора: YOLO26n оптимален для видеопотока в реальном времени, совместим с форматами ONNX/TensorRT и при меньшем числе параметров превосходит предыдущее поколение по точности на целевом домене (сравнение с YOLOv8n — подраздел 3.2.3).

1.4 Обзор методов автоматического распознавания номерных знаков (ANPR)

Типовая ANPR-система включает несколько последовательных этапов [4, 5], схема которых представлена на рисунке 1.1.

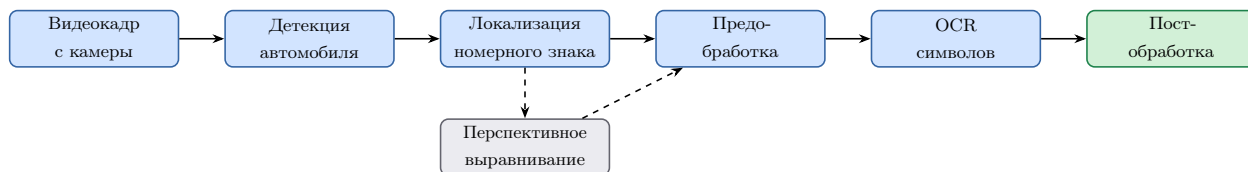


Рисунок 1.1 – Типовая схема ANPR-системы

1.4.1 Сравнение ANPR-подходов

Table 1.2 – Сравнение подходов к ANPR

Подход	Описание	mAP plate	Усл. освещения
Морфология + шаблоны	Пороговая обработка + шаблонный OCR	< 0.70	Плохо
HOG + SVM + Tesseract	HOG-дескриптор для локализации + Tesseract OCR	0.75–0.80	Удовлетворительно
YOLO + CRNN	YOLO-детекция + seq2seq OCR	0.88–0.93	Хорошо
YOLO26 + EasyOCR	Anchor-free, NMS-free детекция + CTC OCR	>0.90	Хорошо

Обоснование выбора: YOLO26n (fine-tuned) + EasyOCR обеспечивает готовую поддержку кириллицы без обучения собственной OCR-модели.

1.5 Обзор методов распознавания марки и типа кузова (VMMR)

Распознавание марки/модели автомобиля (VMMR) относится к задачам тонкой многоклассовой классификации [6, 7, 8]. Задача:

$$\hat{y} = \arg \max_k P(y = k | x), \quad (1.3)$$

где x — изображение, k — класс марки или типа кузова.

Table 1.3 – Сравнение архитектур классификаторов для VMMR

Архитектура	Особенности	Params, М	Top-1 Acc	FPS
AlexNet	Первая глубокая CNN для ImageNet	61	0.74–0.80	высокий
VGG16	Глубокая однородная архитектура	138	0.85–0.89	низкий
ResNet18	Residual connections, лёгкая	11.7	0.88–0.92	высокий
ResNet50	Bottleneck residual, глубже и точнее	25.6	0.90–0.93	средний
EfficientNet-B0	Составной масштаб (глубина/ширина/разрешение)	5.3	0.90–0.94	средний

Обоснование выбора ResNet18: оптимальное соотношение точности и скорости; 11.7М параметров позволяют дообучить модель на ограниченных данных без переобучения.

1.6 Обзор методов определения цвета автомобиля

Цвет автомобиля является значимым визуальным атрибутом для поиска и идентификации транспортных средств [9]. Таблица 1.4 суммирует основные подходы к его классификации.

Table 1.4 – Сравнение методов классификации цвета автомобиля

Метод	Принцип	Ассурасу	Устойчивость к освещению
k-means (RGB)	Кластеризация пикселей	0.70–0.81	Низкая
k-means (HSV)	Кластеризация в HSV-пространстве	0.78–0.85	Средняя
SVM + цветовые гистограммы	Гистограммы + SVM	0.83–0.88	Средняя
CNN (ResNet18)	Сквозное обучение	>0.90	Высокая

Для повышения устойчивости в видеопотоке применяется агрегация предсказаний по нескольким кадрам [10]:

$$\hat{y} = \arg \max_k \sum_{t=1}^T \mathbf{1}[y_t = k]. \quad (1.4)$$

Обоснование выбора: CNN (ResNet18) неявно учится инвариантности к освещению и превосходит кластеризационные методы на 10–15%.

1.7 Цель и задачи работы

Объект исследования — видеоданные с камер дорожного наблюдения, содержащие изображения транспортных средств в реальных условиях эксплуатации.

Предмет исследования — методы и алгоритмы автоматического определения характеристик транспортных средств (номерной знак, цвет кузова, тип кузова) на основе глубокого обучения.

Цель — разработка системы автоматического определения цвета, типа кузова и номерного знака автомобиля по видеопотоку.

Задачи:

1. Провести анализ существующих методов детекции ТС, ANPR, VMMR и определения цвета.
2. Выбрать и обосновать архитектуры моделей для каждой подзадачи.
3. Подготовить обучающие датасеты.

4. Обучить и оценить модели детекции номеров, классификаторы цвета и типа кузова.
5. Интегрировать модели в единый конвейер обработки видео.
6. Провести сравнительный эксперимент ResNet18 vs ResNet50 vs EfficientNet-B0.

Методы исследования: анализ и систематизация научно-технической литературы; методы глубокого обучения (сверточные нейронные сети, transfer learning); экспериментальный метод — сравнительное тестирование архитектур ResNet18, ResNet50 и EfficientNet-B0; метрический анализ качества моделей (mAP, Accuracy, CRR, IoU).

Научно-теоретическая значимость работы состоит в систематизации и сравнительном анализе современных методов детекции транспортных средств, автоматического распознавания номерных знаков, классификации типа кузова и цвета; в обосновании эффективности конвейерной архитектуры на основе transfer learning для задач видеоаналитики транспорта.

Практическая значимость работы состоит в разработке работоспособного прототипа системы, пригодного для встраивания в решения дорожного видеонаблюдения и автоматизации контроля транспортных потоков.

Апробация результатов. Научные публикации по теме работы отсутствуют. Результаты исследования представлены в форме выпускной квалификационной работы.

2.1 Архитектура системы

Разработанная система представляет собой последовательный конвейер, обрабатывающий каждый кадр видеопотока. Общая схема приведена на рисунке 2.1.

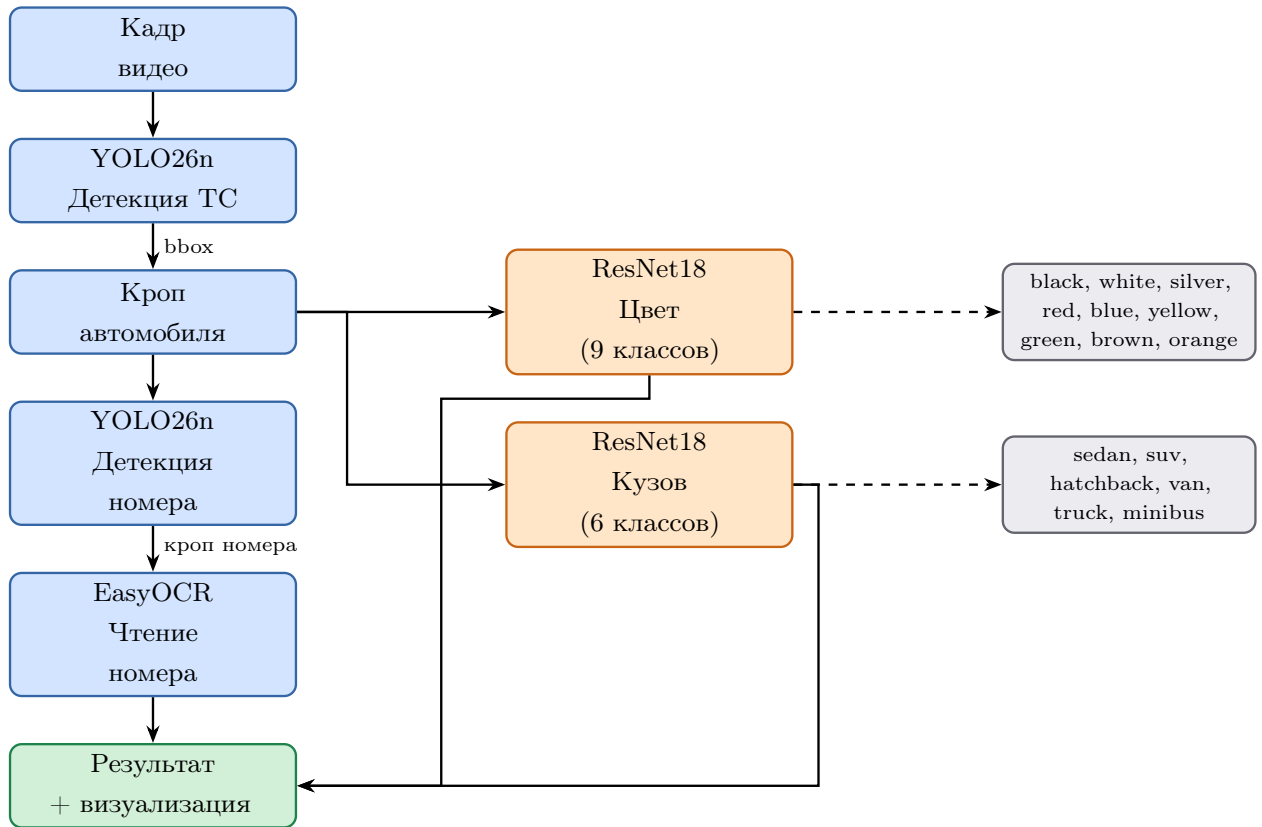


Рисунок 2.1 – Схема конвейера обработки видеокadra

Схема на рисунке 2.1 описывает обработку одиночного кадра. При работе с видеопотоком поверх покадрового конвейера добавляется слой межкадрового сопровождения объектов (трекинг) и агрегации характеристик по каждому транспортному средству; его реализация и обоснование приведены в подразделе 3.5.2.

2.2 Теоретические основы глубокого обучения

Применяемые в работе модели относятся к классу глубоких сверточных нейронных сетей (Deep Convolutional Neural Networks, DCNN). Данный раздел описывает ключевые строительные блоки, необходимые для понимания архитектур YOLO26 и ResNet18.

2.2.1 Сверточные нейронные сети

CNN — специализированный класс нейронных сетей для обработки данных с регулярной сеточной топологией. Три ключевых принципа CNN:

- Разделение весов: одно ядро применяется ко всем позициям входа, что резко сокращает число параметров по сравнению с полносвязным слоем;
- Локальная связность: каждый нейрон реагирует только на рецептивное поле (receptive field) ограниченного размера, что обеспечивает инвариантность к трансляции;

- Иерархия признаков: ранние слои кодируют локальные паттерны (края, текстуры), глубокие — семантические концепции (силуэт кузова, форма номерного знака).

Для одноканального входа $I \in \mathbb{R}^{H \times W}$ и ядра $K \in \mathbb{R}^{m \times n}$ дискретная операция кросс-корреляции (в практике машинного обучения называемая «свёрткой»):

$$(I \star K)[i, j] = \sum_{u=0}^{m-1} \sum_{v=0}^{n-1} I[i+u, j+v] \cdot K[u, v]. \quad (2.1)$$

В многоканальном случае (C_{in} входных каналов, C_{out} выходных) выходная карта признаков:

$$y_k[i, j] = \sum_{c=1}^{C_{\text{in}}} (I_c \star K_{k,c})[i, j] + b_k, \quad k = 1, \dots, C_{\text{out}}, \quad (2.2)$$

где $K_{k,c}$ — ядро k -го фильтра для входного канала c , b_k — смещение (bias).

Число параметров свёрточного слоя: $m \cdot n \cdot C_{\text{in}} \cdot C_{\text{out}} + C_{\text{out}}$, что не зависит от пространственного размера входа $H \times W$. Для сравнения, полносвязный слой потребовал бы $H \cdot W \cdot C_{\text{in}}$ параметров на каждый выходной нейрон.

2.2.2 Функции активации

Для введения нелинейности после каждого свёрточного слоя применяется функция активации. В ResNet18 используется ReLU:

$$\text{ReLU}(x) = \max(0, x). \quad (2.3)$$

ReLU не насыщается на положительной полуоси, градиент равен единице для $x > 0$ — это ключевое свойство, предотвращающее затухание градиентов при прямолинейном прохождении.

В YOLO26 применяется SiLU (Sigmoid Linear Unit, также известная как Swish):

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}, \quad (2.4)$$

обеспечивающая плавный градиент вблизи нуля и улучшающая сходимость глубоких детекционных сетей.

2.2.3 Pooling и агрегация признаков

Max-pooling с окном $k \times k$ и шагом s уменьшает пространственное разрешение, сохраняя наиболее значимые активации:

$$y[i, j] = \max_{\substack{u \in [0, k) \\ v \in [0, k)}} x[si + u, sj + v]. \quad (2.5)$$

Global Average Pooling (GAP) применяется в конце классификационных сетей перед FC-слоем:

$$\text{GAP}_c = \frac{1}{H \cdot W} \sum_{i=1}^H \sum_{j=1}^W x_c[i, j]. \quad (2.6)$$

GAP сводит пространственную карту к одному числу на канал, что обеспечивает инвариантность к размеру входного изображения и значительно снижает число параметров по сравнению с flatten + FC. В ResNet18 GAP даёт вектор $\mathbb{R}^{512}$ из feature map $7 \times 7 \times 512$.

2.2.4 Пакетная нормализация

Пакетная нормализация (Batch Normalization, BN) нормирует активации по мини-батчу:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}, \quad y_i = \gamma \hat{x}_i + \beta, \quad (2.7)$$

где $\mu_{\mathcal{B}}$ и $\sigma_{\mathcal{B}}^2$ — выборочные среднее и дисперсия по батчу, γ и β — обучаемые параметры масштабирования и сдвига, $\varepsilon \approx 10^{-5}$ — стабилизирующая константа.

Основные эффекты BN:

1. снижение чувствительности к выбору начальной LR;
2. неявная регуляризация — уменьшает необходимость в Dropout;
3. ускорение сходимости в 10–14 раз по сравнению с сетями без BN при одинаковом расписании LR.

BN присутствует в каждом convolution-блоке как ResNet18, так и YOLO26, что обусловило выбор относительно высоких начальных LR в обоих случаях.

2.3 Детекция транспортных средств: YOLO26

2.3.1 Внутренняя архитектура YOLO26

YOLO26 [3] — актуальное поколение одноэтапных детекторов Ultralytics (2026). Как и предшественники, модель состоит из трёх функциональных блоков (рисунок 2.2): backbone для извлечения признаков, neck для их многомасштабного слияния и head для предсказания боксов и классов. Принципиальные отличия YOLO26 от предыдущих поколений сосредоточены не в топологии, а в способе вывода и обучения:

- End-to-end инференс без NMS — модель напрямую выдаёт финальный набор детекций, исключая постобработку Non-Maximum Suppression и связанную с ней задержку;
- отказ от Distribution Focal Loss (DFL) — координаты бокса регрессируются напрямую, что упрощает экспорт в ONNX/TensorRT и ускоряет вывод на CPU;
- ProgLoss (Progressive Loss Balancing) — динамическая балансировка компонентов функции потерь для устойчивого обучения;
- STAL (Small-Target-Aware Label assignment) — назначение меток с учётом мелких объектов, повышающее их полноту (recall);
- оптимизатор MuSGD — гибрид Muon и SGD с более стабильной и быстрой сходимостью.

Эти изменения дают на COCO mAP@0.50:0.95 = 40.9 для версии nano против 37.3 у YOLOv8n при меньшем числе параметров (2.4M против 3.2M) и FLOPs (5.4 против 8.7 GFLOPs).

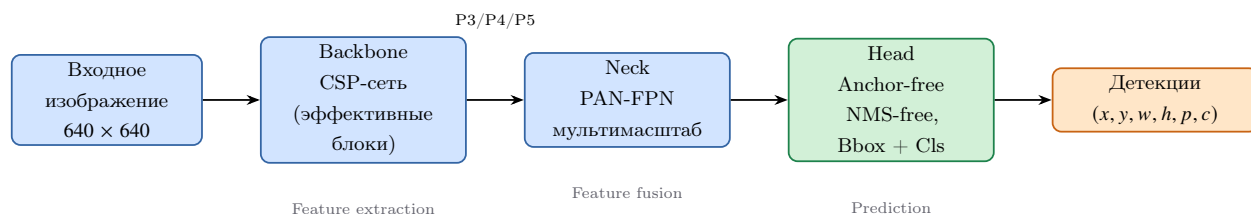


Рисунок 2.2 – Упрощённая схема архитектуры YOLO26

Топология backbone–neck–head унаследована от предыдущих поколений семейства; ниже подробно рассматриваются backbone (подраздел 2.3.3), neck (подраздел 2.3.4), anchor-free голова с end-to-end инференсом без NMS (подраздел 2.3.5), а также функция потерь и оптимизатор (подраздел 2.3.6).

2.3.2 Эволюция семейства YOLO

Семейство YOLO прошло путь от первой версии (2016) до YOLO26 (2026), последовательно устраняя ключевые ограничения предыдущих поколений.

Table 2.1 – Эволюция ключевых характеристик архитектур YOLO

Версия	Год	Ключевые нововведения	mAP@0.5
YOLOv1	2016	Сетка $S \times S$; B боксов на ячейку; softmax классов	63.4
YOLOv2	2017	Anchor-boxes, BatchNorm, многомасштабное обучение	78.6
YOLOv3	2018	Darknet-53; детекция на 3 масштабах; sigmoid классификаторы	82.1
YOLOv4	2020	CSPDarknet53, PANet, CIoU loss, Mosaic-аугментация	84.0
YOLOv5	2020	AutoAnchor, Focus-слой, адаптивный batching (PyTorch)	84.7
YOLOv6	2022	RepVGG backbone, знаниевая дистилляция	85.0
YOLOv7	2022	Extended-ELAN, составное масштабирование	85.6
YOLOv8	2023	Anchor-free, C2f блоки, decoupled head, DFL loss	86.3
YOLO26	2026	End-to-end (NMS-free), отказ от DFL, ProgLoss + STAL, MuSGD	—

*Для YOLO26 в столбце mAP приведён прочерк, так как характеристика измеряется иначе: на COCO версия nano даёт mAP@0.5:0.95 = 40.9 против 37.3 у YOLOv8n [3].

Переход к anchor-free подходу произошёл на поколении 2023 года и унаследован YOLO26. В классических anchor-based детекторах (YOLOv2–v5) необходимо предварительно кластеризовать размеры объектов в датасете для выбора наборов якорей. Anchor-free голова вместо этого напрямую предсказывает геометрию бокса, что упрощает адаптацию к нестандартным доменам (например, номерные знаки имеют специфическое соотношение сторон $\approx 4:1$, плохо покрываемое универсальными COCO-якорями). YOLO26 развивает эту линию: помимо отказа от якорей, модель убирает и постобработку NMS, выполняя детекцию end-to-end (подраздел 2.3.5).

2.3.3 Backbone: CSP-сеть

Backbone YOLO26 основан на принципе Cross-Stage Partial Networks (CSP). CSP разделяет входной тензор на две части: одна проходит через набор свёрточных блоков (dense path), другая — напрямую (identity path), после чего результаты конкатенируются и объединяются 1×1 свёрткой:

$$\text{CSP}(X) = \text{Conv}_{1 \times 1} \left(\text{Concat} \left[\text{DenseBlock}(X_{C/2}), X_{C/2} \right] \right), \quad (2.8)$$

где $X_{C/2}$ и $X_{C/2}$ — первая и вторая половины каналов тензора $X \in \mathbb{R}^{C \times H \times W}$. Это снижает FLOPS примерно на 30% при сохранении богатого градиентного потока через identity path.

CSP-блоки с несколькими bottleneck-ветвями (развитие подхода C3→C2f из поколений 2020–2023 годов) сохраняют все промежуточные feature maps в финальной конкатенации, что увеличивает число независимых путей градиента при незначительном росте параметров. В YOLO26 backbone дополнительно оптимизирован под сокращение FLOPs: версия nano использует лишь 5.4 GFLOPs (против 8.7 у YOLOv8n) при более высокой точности на COCO.

2.3.4 Neck: Path Aggregation Network

Для обнаружения объектов разных масштабов (мелкие — номерные знаки вдали, крупные — автомобили вблизи) YOLO26 использует PAN-FPN (Path Aggregation Network + Feature Pyramid Network).

FPN (top-down path): строит пирамиду признаков, обогащая поверхностные (детальные) уровни

семантикой глубоких (абстрактных) уровней:

$$P_k = \text{Conv}(\text{Upsample}(P_{k+1}) + C_k), \quad (2.9)$$

где C_k — выход backbone на уровне k , P_k — обогащённая feature map.

PAN (bottom-up path): добавляет восходящий путь, усиливающий точность пространственной локализации за счёт низкоуровневых (высокого разрешения) признаков:

$$N_k = \text{Conv}(\text{Stride-Conv}(N_{k-1}) + P_k). \quad (2.10)$$

Для входа 640×640 три детекционных головы обрабатывают:

- P3 (80×80) — мелкие объекты (далёкие номерные знаки);
- P4 (40×40) — объекты среднего масштаба;
- P5 (20×20) — крупные объекты (автомобили вблизи).

2.3.5 Anchor-free голова и end-to-end инференс без NMS

YOLO26 напрямую предсказывает расстояния от центра ячейки до четырёх сторон бокса (l, t, r, b) , из которых восстанавливается ограничивающий прямоугольник:

$$x_1 = x_c - l, \quad y_1 = y_c - t, \quad x_2 = x_c + r, \quad y_2 = y_c + b. \quad (2.11)$$

Отказ от Distribution Focal Loss. В поколении 2023 года координаты регрессировались через DFL — предсказание дискретного распределения вероятностей по бинам с последующим взятием математического ожидания. YOLO26 отказывается от DFL и регрессирует координаты напрямую. Это убирает дополнительную ветвь в голове и упрощает граф вычислений: ускоряется инференс (по данным Ultralytics до 43% на CPU) и устраняются проблемы при экспорте в ONNX/TensorRT, где слой DFL ранее требовал отдельной поддержки.

End-to-end инференс без NMS. Классические детекторы (в том числе YOLOv8) выдают для каждого объекта множество перекрывающихся боксов, которые затем прореживаются алгоритмом Non-Maximum Suppression (NMS) по порогу IoU. NMS — непараллелизуемая постобработка с собственной задержкой и гиперпараметром (порогом IoU). YOLO26 обучается с назначением меток «один-к-одному», благодаря чему голова сразу выдаёт финальный непересекающийся набор детекций без NMS. Это снижает сквозную задержку и делает время инференса детерминированным — не зависящим от числа кандидатов в кадре.

2.3.6 Функция потерь и оптимизация

После отказа от DFL функция потерь YOLO26 объединяет два компонента — локализацию и классификацию:

$$\mathcal{L} = \lambda_{\text{box}} \mathcal{L}_{\text{box}} + \lambda_{\text{cls}} \mathcal{L}_{\text{cls}}. \quad (2.12)$$

$\mathcal{L}_{\text{box}}$ — Complete IoU (CIoU) loss учитывает три геометрических аспекта совпадения боксов:

$$\mathcal{L}_{\text{CIoU}} = 1 - \text{IoU} + \frac{\rho^2(b, b^{\text{gt}})}{c^2} + \alpha v, \quad (2.13)$$

где $\rho^2(b, b^{\text{gt}})$ — квадрат евклидова расстояния между центрами предсказанного и эталонного боксов; c — диагональ минимального объемлющего прямоугольника;

$$v = \frac{4}{\pi^2} \left(\arctan \frac{w^{\text{gt}}}{h^{\text{gt}}} - \arctan \frac{w}{h} \right)^2 \quad (2.14)$$

— штраф за разницу соотношений сторон; $\alpha = v / (1 - \text{IoU} + v)$ — балансирующий коэффициент. В отличие от простого IoU-loss, CIoU даёт ненулевой градиент при непересекающихся боксах и сходится быстрее.

$\mathcal{L}_{\text{cls}}$ — Binary Cross-Entropy (BCE) применяется независимо к каждому классу через сигмоид-активацию:

$$\mathcal{L}_{\text{cls}} = - \sum_{k=1}^K [y_k \log \hat{p}_k + (1 - y_k) \log(1 - \hat{p}_k)], \quad (2.15)$$

где $\hat{p}_k = \sigma(z_k)$ — предсказанная вероятность класса k .

Дополнительно YOLO26 вводит два механизма обучения:

- ProgLoss (Progressive Loss Balancing) — прогрессивно балансирует вклад $\mathcal{L}_{\text{box}}$ и $\mathcal{L}_{\text{cls}}$ по ходу обучения, стабилизируя сходимость на ранних эпохах;
- STAL (Small-Target-Aware Label assignment) — назначение положительных меток с учётом мелких объектов, повышающее их полноту (recall); это релевантно для удалённых номерных знаков.

Оптимизатор MuSGD сочетает свойства Muon и SGD, обеспечивая более стабильную и быструю сходимость, чем чистый SGD; в Ultralytics он используется как оптимизатор по умолчанию для YOLO26.

2.3.7 Параметры инференса

Table 2.2 – Параметры запуска YOLO26n для детекции ТС

Параметр	Значение
Модель	yolo26n.pt (предобученная, COCO)
Порог уверенности	$p_{\min} = 0.40$
Постобработка	end-to-end, без NMS
Используемые классы	2 (car), 3 (motorcycle), 5 (bus), 7 (truck)
Размер входа	640×640
Аппаратная платформа	GPU NVIDIA Tesla T4 / CPU

2.4 Детекция номерных знаков: дообученный YOLO26n

2.4.1 Схема transfer learning

Процесс дообучения показан на рисунке 2.3.

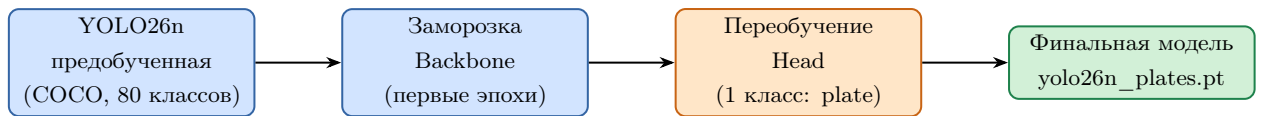


Рисунок 2.3 – Схема transfer learning для детектора номерных знаков

2.4.2 Параметры обучения

Table 2.3 – Гиперпараметры дообучения детектора номеров

Параметр	Значение
Базовая модель	yolo26n.pt
Эпохи	50
Размер входа	640×640
Размер батча	16
Оптимизатор	optimizer=auto (Ultralytics)
Начальная LR	подбирается автоматически
Weight decay	5×10^{-4}
Число классов	1 (license_plate)
Метрика сохранения	mAP@0.50 на тестовой выборке

2.5 Оптическое распознавание номеров: EasyOCR

EasyOCR использует двухэтапный конвейер, представленный на рисунке 2.4.

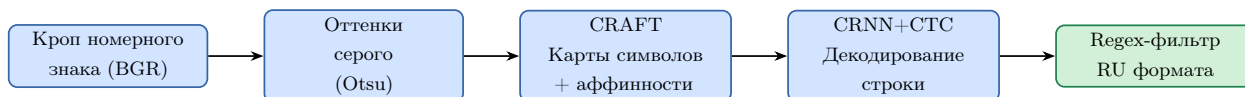


Рисунок 2.4 – Конвейер EasyOCR для распознавания номерного знака

Качество OCR оценивается через Character Error Rate:

$$\text{CER} = \frac{S + D + I}{N}, \quad (2.16)$$

где S, D, I — замены, удаления, вставки (алгоритм Левенштейна), N — длина эталонной строки.

2.6 Классификация цвета и типа кузова: ResNet18

2.6.1 Архитектура ResNet18

Ключевая идея ResNet — shortcut-соединения:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}, \quad (2.17)$$

что устраняет затухание градиентов и позволяет строить глубокие сети. Схема residual-блока и размещение слоёв в ResNet18 приведены на рисунке 2.5.

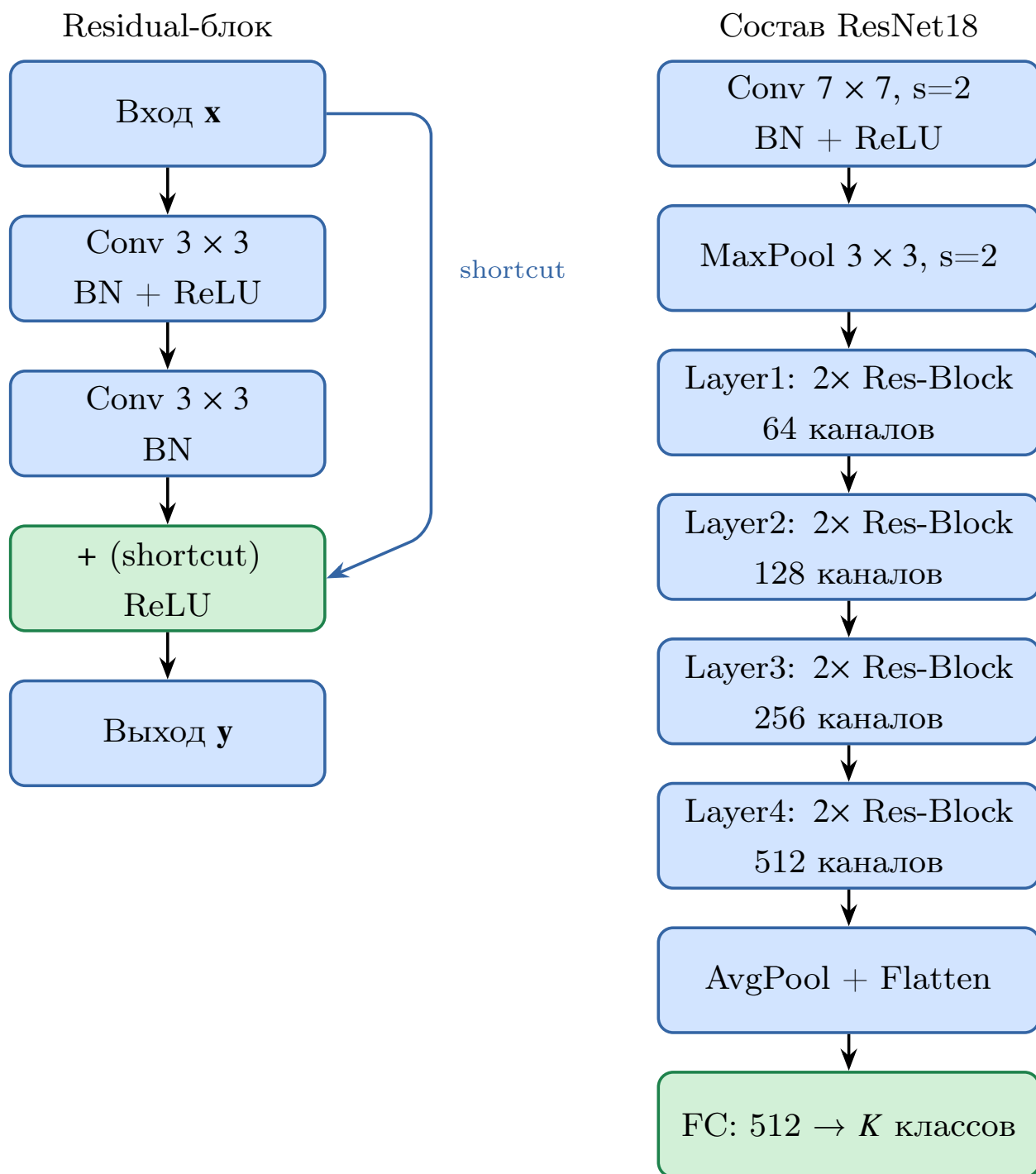


Рисунок 2.5 – Residual-блок и структура ResNet18 (18 слоёв, K — число классов)

2.6.2 Двухфазное дообучение

Стратегия двухфазного обучения показана на рисунке 2.6.

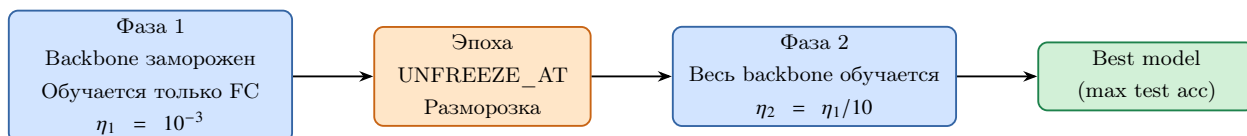


Рисунок 2.6 – Стратегия двухфазного дообучения ResNet18

Функция потерь — кросс-энтропия:

$$\mathcal{L}_{\text{CE}} = - \sum_{k=1}^K y_k \log \hat{y}_k, \quad (2.18)$$

где y_k — one-hot метка, $\hat{y}_k$ — предсказанная вероятность.

2.6.3 Проблема затухающего градиента в глубоких сетях

При обучении методом обратного распространения ошибки (backpropagation) производная функции потерь по активациям слоя l вычисляется через цепное правило:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_L} \prod_{k=l}^{L-1} \frac{\partial \mathbf{x}_{k+1}}{\partial \mathbf{x}_k}, \quad (2.19)$$

где $\mathbf{x}_k$ — активации слоя k , L — глубина сети.

Произведение якобианов $\prod_{k=l}^{L-1} J_k$ экспоненциально убывает по числу слоёв при $\|J_k\| < 1$: нижние слои получают практически нулевые градиенты и фактически прекращают обучаться. Это явление называется затухающим градиентом и ограничивало практически применимую глубину сетей значением ≈ 20 слоёв до появления ResNet (He et al., 2015).

Попытки строить сети глубиной 56–110 слоёв (plain networks) до ResNet приводили к парадоксу: добавление слоёв ухудшало тренировочную ошибку — не из-за переобучения, а из-за неудовлетворительной оптимизации.

2.6.4 Теоретическое обоснование residual connections

Ключевая гипотеза He et al. (2015): вместо обучения прямого отображения $\mathcal{H}(\mathbf{x})$ легче выучить остаточное (residual) отображение $\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}. \quad (2.20)$$

Если оптимальное отображение близко к тождественному, достаточно обнулить $\mathcal{F} \approx \mathbf{0}$ — что тривиально для стека нелинейных слоёв.

Анализ градиентного потока. Производная потерь через shortcut-соединения:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_L} \underbrace{\prod_{k=l}^{L-1} \left(1 + \frac{\partial \mathcal{F}_k}{\partial \mathbf{x}_k} \right)}_{\text{не обращается в нуль}}. \quad (2.21)$$

Единица в каждом множителе гарантирует, что произведение не стремится к нулю независимо от глубины. Это теоретически обосновывает возможность обучения ResNet глубиной от 18 до 1000 слоёв.

Выбор ResNet18 vs. более глубоких вариантов: При ограниченных датасетах (<50k изображений) более лёгкие модели часто превосходят ResNet50/101 за счёт лучшей регуляризации. Для VCoR ($\approx 15k$ изображений) и CompCars ($\approx 30k$) ResNet18 оказывается оптимальным компромиссом между ёмкостью и переобучением.

2.6.5 ImageNet-предобучение и стратегии transfer learning

Предобученные на ImageNet (1.28M изображений, 1000 классов) веса кодируют универсальные визуальные признаки трёх уровней:

- layer1–layer2: края, ориентации, текстуры;
- layer3: части объектов (круглые формы, угловые элементы);

- layer4: семантические концепции — здесь кодируется специфика «автомобильного» домена.

Применяются три стратегии transfer learning:

1. Feature extraction (заморозка всего backbone): обучается только FC-голова. Быстро, но ограничено — предобученные признаки могут не соответствовать целевому домену.
2. Partial fine-tuning: размораживаются верхние слои backbone (layer3+layer4+FC). Применяется в первых экспериментах с классификатором кузова.
3. Full fine-tuning: обучается весь backbone с LR в 10 раз ниже, чем для FC. Применяется для классификатора цвета (фаза 2). Требует достаточного объёма целевых данных.

Обоснование двухфазной стратегии (фаза 1: заморозка, фаза 2: разморозка): в начале дообучения FC-веса случайны и порождают большие градиенты. Если одновременно обучать backbone, эти большие градиенты разрушат предобученные признаки (catastrophic forgetting). Заморозка backbone на первых эпохах стабилизирует FC, после чего backbone размораживается с малой LR — это позволяет тонко адаптировать признаки к целевому домену (изображения автомобилей с дорожных камер).

2.6.6 Гиперпараметры обучения

Table 2.4 – Гиперпараметры обучения классификаторов ResNet18

Параметр	Классификатор цвета	Классификатор кузова
Число классов	9	6
Макс. эпох	20	40 (early stop)
Разморозка	эп. 5 (весь backbone)	с эп. 6 (layer3+layer4+fc)
Батч	32	32
LR	$10^{-3} \rightarrow 10^{-4}$	5×10^{-5}
Оптимизатор	Adam	Adam
β_1, β_2	0.9, 0.999	0.9, 0.999
StepLR шаг	5 эпох	—
StepLR γ	0.3	—
Early stopping	—	patience = 7
Размер входа	224×224	224×224

2.6.7 Аугментация данных

Table 2.5 – Аугментации для обучения классификаторов

Преобразование	Параметры	Назначение
Обучающая выборка		
RandomResizedCrop	224 , scale $[0.7, 1.0]$	Масштаб и позиция
RandomHorizontalFlip	$p = 0.5$	Зеркальное отражение
ColorJitter	brightness=0.3, contrast=0.3	Освещение
RandomRotation	± 10	Поворот
Normalize	$\mu = [.485, .456, .406]$	ImageNet stats
Валидационная выборка		
Resize	224×224	Размер входа
Normalize	$\mu = [.485, .456, .406]$	ImageNet stats

2.7 Алгоритмы оптимизации

2.7.1 Стохастический градиентный спуск с импульсом

Базовый SGD обновляет параметры по градиенту на мини-батче $\mathcal{B}_t$:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t; \mathcal{B}_t), \quad (2.22)$$

где η — скорость обучения (learning rate).

SGD с импульсом (momentum) накапливает «инерцию» в направлении постоянного градиента, ускоряя сходимость:

$$v_{t+1} = \mu v_t + \nabla_{\theta} \mathcal{L}(\theta_t; \mathcal{B}_t), \quad \theta_{t+1} = \theta_t - \eta v_{t+1}, \quad (2.23)$$

где μ — коэффициент импульса (типичное значение для детекторов семейства YOLO — $\mu = 0.937$).

Выбор оптимизатора для детектора YOLO26: Детектор номеров обучался в режиме optimizer=auto библиотеки Ultralytics, которая для датасетов объёмом менее 10 000 изображений автоматически подбирает алгоритм оптимизации и начальную скорость обучения. При этом для YOLO26 в качестве базового алгоритма введён MuSGD — гибрид Muon и SGD, дающий более стабильную и быструю сходимость, чем чистый SGD с импульсом. Автоматический подбор при ограниченном объёме целевых данных и коротком расписании обучения обеспечивает устойчивую сходимость без ручной настройки скорости обучения.

2.7.2 Adam

Adam (Adaptive Moment Estimation) адаптирует LR для каждого параметра, поддерживая первый и второй моменты градиентов:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (2.24)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad (2.25)$$

где $g_t = \nabla_{\theta} \mathcal{L}_t$. Скорректированные (debiased) оценки:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}. \quad (2.26)$$

Обновление параметров:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \varepsilon} \hat{m}_t, \quad (2.27)$$

где $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$.

Обоснование Adam для классификаторов: Adam быстро сходится за 10–30 эпох и менее чувствителен к выбору начальной LR. При fine-tuning предобученных сетей адаптивность Adam предотвращает излишнее обновление слоёв, которые уже содержат хорошие признаки ImageNet.

2.7.3 Планировщики скорости обучения

StepLR снижает LR в γ раз каждые S эпох:

$$\eta_t = \eta_0 \cdot \gamma^{\lfloor t/S \rfloor}. \quad (2.28)$$

Для классификатора цвета: $S = 5$, $\gamma = 0.3$, что даёт убывающую последовательность: $\eta_0 = 10^{-3} \rightarrow 3.4 \times 10^{-4} \rightarrow 10^{-4} \rightarrow \dots$. Постепенное снижение позволяет крупному шагу обеспечить быстрый прогресс в начале, а малому — точную настройку вблизи оптимума.

Линейное затухание LR применяется при обучении детектора (режим cos_lr=False по

умолчанию, $\text{lrf}=0.01$):

$$\eta_t = \eta_0 \left(1 - (1 - \text{lrf}) \frac{t}{T} \right), \quad (2.29)$$

то есть LR линейно убывает от η_0 до $\eta_0 \cdot \text{lrf}$ к концу обучения. Плавное снижение скорости обучения позволяет детектору сойтись в устойчивый минимум потерь.

2.8 Методы регуляризации

Регуляризация — совокупность методов, снижающих переобучение (overfitting) и улучшающих обобщающую способность модели.

2.8.1 L2-регуляризация (weight decay)

К функции потерь добавляется штраф на ℓ_2 -норму весов:

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \frac{\lambda}{2} \|\theta\|_2^2. \quad (2.30)$$

В данной работе $\lambda = 5 \times 10^{-4}$ для детектора номеров. Weight decay ограничивает «свободу» модели и эквивалентен MAP-оценке с гауссовым априором на веса $\theta \sim \mathcal{N}(0, 1/\lambda)$.

2.8.2 Ранняя остановка

Если контрольная метрика не улучшается в течение patience эпох, обучение прекращается, а лучшая (по тестовой метрике) модель сохраняется:

$$\text{stop if } e - e_{\text{best}} > \text{patience}, \quad (2.31)$$

где e_{best} — номер эпохи с наилучшей метрикой.

В данной работе early stopping с patience=7 применяется для классификатора кузова, что предотвратило переобучение — лучшая модель получена на эпохе 27, обучение остановлено на эпохе 34.

2.8.3 Аугментация как регуляризация

Аугментация увеличивает эффективный объём обучающей выборки без сбора новых данных. Математически оптимизируется:

$$\min_{\theta} \mathbb{E}_{(x,y) \sim \mathcal{D}} \mathbb{E}_{t \sim \mathcal{T}} \mathcal{L}(f_{\theta}(t(x)), y), \quad (2.32)$$

где $\mathcal{T}$ — семейство реалистичных трансформаций.

Аугментации в таблице 2.5 подобраны с учётом реалистичности для видеопотока с дорожных камер:

- RandomHorizontalFlip: автомобили въезжают с обеих сторон кадра — зеркальные копии равновероятны;
- ColorJitter (brightness=0.3, contrast=0.3): освещение меняется в течение суток и при смене погоды;
- RandomRotation ± 10 : небольшой наклон возможен при установке камеры под углом;
- RandomResizedCrop (scale [0.7, 1.0]): имитирует различное расстояние до автомобиля; нижняя граница 0.7 выбрана так, чтобы сохранить достаточно цветовой информации кузова.

Набор аугментаций различается для двух задач. Для классификатора цвета применяется базовый набор (таблица 2.5) с сохранением цветовых искажений (ColorJitter), ключевых для устойчивости к освещению. Для классификатора типа кузова тот же набор усилен под доменный разрыв CompCars $\rightarrow$ улица: добавлены RandomPerspective (distortion 0.3, $p = 0.5$) — имитация уличных ракурсов 3/4; GaussianBlur ($p = 0.25$) — низкое разрешение и смаз удалённых кадров; RandomErasing

($p = 0.3$) — частичные перекрытия. ColorJitter при этом сохранён (насыщенность снижена до 0.15), нижняя граница RandomResizedCrop — 0.72, а перед FC-слоем добавлены Dropout 0.5 и сглаживание меток (label smoothing 0.1) как дополнительная регуляризация.

2.8.4 Mixup-аугментация

Стандартные методы аугментации (отражения, кропы, цветовые искажения) преобразуют каждый пример независимо, не затрагивая метки. Mixup [11] расширяет это понятие: он формирует синтетические примеры как выпуклые комбинации двух случайно выбранных примеров из обучающей выборки вместе с их метками.

Мотивация. Метод Mixup вытекает из минимизации вицинального риска (Vicinal Risk Minimization, VRM) — обобщения принципа эмпирической минимизации риска (ERM). В VRM модель обучается не на точечных примерах, а на окрестности каждого примера в пространстве входов:

$$R_{\text{vic}}(f) = \mathbb{E}_{(\tilde{x}, \tilde{y}) \sim \tilde{P}} [\mathcal{L}(f(\tilde{x}), \tilde{y})], \quad (2.33)$$

где $\tilde{P}$ — вициональное распределение, индуцированное обучающей выборкой. Mixup задаёт $\tilde{P}$ через попарное смешивание примеров.

Формализация. Для каждой пары (x_i, y_i) и (x_j, y_j) из мини-батча смешанный пример строится как:

$$\tilde{x} = \lambda x_i + (1 - \lambda) x_j, \quad \tilde{y} = \lambda y_i + (1 - \lambda) y_j, \quad \lambda \sim \text{Beta}(\alpha, \alpha), \quad (2.34)$$

где $\lambda \in [0, 1]$ — коэффициент смешивания. Метки $\tilde{y}$ при этом становятся мягкими (soft labels): для задачи классификации y_i — one-hot вектор, и $\tilde{y}$ представляет взвешенную вероятностную смесь двух классов. Функция потерь на смешанном примере принимает вид:

$$\mathcal{L}(\tilde{x}, \tilde{y}) = \lambda \text{CE}(f(\tilde{x}), y_i) + (1 - \lambda) \text{CE}(f(\tilde{x}), y_j). \quad (2.35)$$

Свойства распределения $\text{Beta}(\alpha, \alpha)$. При $\alpha = 1$ получаем равномерное распределение $\lambda \sim U[0, 1]$ — максимальное смешивание. При $\alpha \rightarrow 0$ распределение вырождается в точечную меру на $\{0, 1\}$ (аугментация сводится к случайному выбору одного из двух примеров). При $\alpha < 1$ плотность $\text{Beta}(\alpha, \alpha)$ бимодальна: масса сосредоточена у краёв 0 и 1, то есть большинство смешанных примеров «почти чистые». При $\alpha > 1$ плотность унимодальна с максимумом в $\lambda = 0.5$: примеры сильно смешаны. Этим объясняется выбор рабочего значения $\alpha \in [0.1, 0.4]$ в большинстве задач: достаточно регуляризующее действие при сохранении осмысленного обучающего сигнала.

Регуляризующий эффект. Mixup действует в трёх взаимосвязанных механизмах:

1. Сглаживание меток (label smoothing): мягкие метки снижают уверенность модели в граничных областях и уменьшают переобучение на шумовые примеры;
2. Линеаризация скрытых представлений: обучение на линейных интерполяциях стимулирует модель строить линейно-разделимые представления между классами, что улучшает обобщение на незнакомые примеры;
3. Явное обучение на граничных примерах: смешанные примеры $(x_i + x_j)/2$ лежат в окрестности решающей границы — модель явно обучается на переходных случаях, наиболее трудных при тестировании.

Особенность метрики на обучении. При Mixup train accuracy, вычисленная как $\arg \max$ предсказания против hard label одного из смешанных примеров, оказывается систематически заниженной относительно val accuracy: модель оценивается на смешанных изображениях с мягкими метками, тогда как валидация проводится на чистых примерах. Это ожидаемое поведение и не является признаком underfitting.

2.8.5 Test-Time Augmentation (TTA)

В отличие от обучающих аугментаций и Mixup, действующих на этапе обучения, Test-Time Augmentation повышает точность на этапе инференса без дообучения модели. Изображение пропускается через сеть в нескольких аугментированных вариантах, а итоговое распределение вероятностей усредняется. Для классификатора типа кузова применяется TTA по горизонтальному отражению:

$$\hat{p}(x) = \frac{1}{2} \left(\text{softmax}(f(x)) + \text{softmax}(f(\text{flip}(x))) \right), \quad (2.36)$$

где $\text{flip}(x)$ — зеркально отражённое по горизонтали изображение. Усреднение по двум видам снижает дисперсию предсказаний и устраняет случайные ошибки, чувствительные к ориентации объекта. Приём вычислительно дешёв (две прямых прогонки вместо одной) и не требует изменения весов; в эксперименте он дал прирост Ассурасы top-1 около 0.5–1.0 п.п.

2.9 Сравнимые архитектуры: ResNet50 и EfficientNet-B0

Для объективной оценки качества предобученного backbone классификатор ResNet18 сравнивается с двумя альтернативами: более глубокой ResNet50 (bottleneck-блоки, 23.5 М параметров) и EfficientNet-B0 (составное масштабирование, 4.0 М параметров). Базовым строительным блоком EfficientNet-B0 является MBConv — инвертированный residual-блок с depthwise-свёрткой и squeeze-excitation (рисунок 2.7): входные карты признаков сначала расширяются свёрткой 1×1 , обрабатываются depthwise-свёрткой 3×3 и проецируются обратно свёрткой 1×1 со skip-соединением.

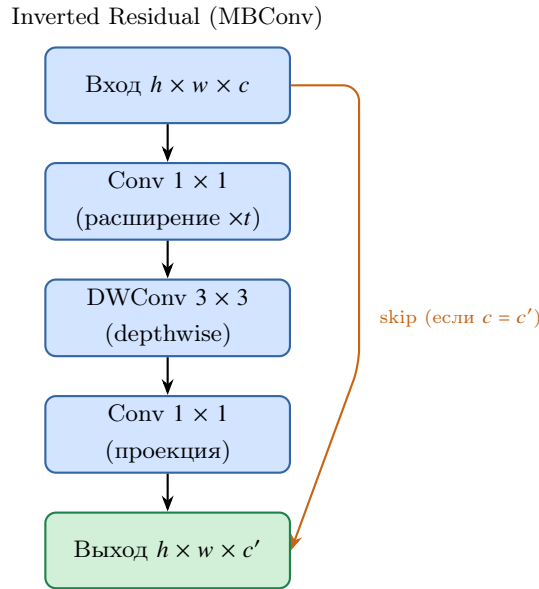


Рисунок 2.7 – Инвертированный residual-блок (MBConv) — основной блок EfficientNet-B0

Table 2.6 – Сравнимые архитектуры классификатора типа кузова

Характеристика	ResNet18	ResNet50	EfficientNet-B0
Params, M	11.2	23.5	4.0
Blocks	Residual	Bottleneck	MBConv (inv. res. + SE)
Conv type	Standard	Standard	Depthwise sep.
FLOPS (224)	1.8G	4.1G	0.39G

2.9.1 ResNet50: bottleneck-архитектура

ResNet50 [12] углубляет идею residual-обучения за счёт bottleneck-блоков. В отличие от базового блока ResNet18 из двух свёрток 3×3 , bottleneck-блок применяет последовательность $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$: первая свёртка 1×1 снижает число каналов (сжатие), свёртка 3×3 обрабатывает сжатое представление, а финальная 1×1 восстанавливает размерность. Такая схема позволяет нарастить глубину до 50 слоёв при умеренной вычислительной стоимости (4.1G FLOPS, 25.6 М параметров в исходной ImageNet-версии). Большая глубина формирует более богатую иерархию признаков, что в проведённом эксперименте (подраздел 3.4.4) выразилось в наивысшей точности в режиме feature extraction (0.666). Платой выступает практически двукратное увеличение задержки инференса по сравнению с ResNet18.

2.9.2 EfficientNet-B0: составное масштабирование

EfficientNet [13] основан на принципе составного масштабирования (compound scaling): глубина d , ширина w и разрешение входа r масштабируются согласованно через единый коэффициент ϕ :

$$d = \alpha^\phi, \quad w = \beta^\phi, \quad r = \gamma^\phi, \quad \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2, \quad (2.37)$$

где константы α, β, γ найдены поиском по сетке на базовой модели. EfficientNet-B0 построена из блоков MBConv (рисунок 2.7) с модулем squeeze-excitation (SE) [14], который адаптивно перевзвешивает каналы признаков:

$$s_c = \sigma(W_2 \delta(W_1 z))_c, \quad z_c = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W u_c(i, j), \quad (2.38)$$

где z_c — глобальный average pooling по каналу c , δ — ReLU, σ — сигмоида, а выход $s_c \in [0, 1]$ масштабирует исходную карту признаков u_c . Благодаря depthwise-свёрткам и SE-механизму EfficientNet-B0 достигает конкурентной точности при наименьшем числе параметров (4.0 М, 0.39G FLOPS). Однако depthwise-операции обладают низкой арифметической интенсивностью и плохо утилизируют параллельные блоки GPU, поэтому на T4 модель показала наименьший FPS среди трёх архитектур (подраздел 3.4.4).

2.10 Датасеты

Table 2.7 – Сводная информация об используемых датасетах

Датасет	Задача	Источник	Изобр.	Классов	Формат
COCO 2017	Детекция ТС	COCO/Ultralytics	118k	80 (4 для ТС)	YOLO
Russian license plates	Детекция номеров	Kaggle (tomilovivan)	668	1	YOLO txt
NOMEROFF-NET RU	Оценка OCR	nomeroff.net.ua	2845	—	JSON ann.
VCoR	Цвет авто	Kaggle	$\approx 15k$	9	ImageFolder
CompCars body-type	Тип кузова	CUNK	$\approx 30k$	6	ImageFolder

2.10.1 VCoR — классификация цвета

Table 2.8 – Классы датасета VCoR и соответствующие цвета

#	Класс	Приблизительный RGB	Hex
1	black	(25, 25, 25)	#191919
2	white	(230, 230, 230)	#E6E6E6
3	silver	(175, 180, 185)	#AFB4B9
4	red	(195, 30, 30)	#C31E1E
5	blue	(30, 60, 200)	#1E3CC8
6	yellow	(220, 200, 20)	#DCC814
7	green	(30, 145, 30)	#1E911E
8	brown	(100, 60, 30)	#643C1E
9	orange	(215, 105, 20)	#D7691E

2.10.2 CompCars — классификация типа кузова

Table 2.9 – Маппинг оригинальных классов CompCars на целевые

Оригинальный класс CompCars	Целевой класс	Описание
sedan, fastback	sedan	Легковой, 4 двери
hatchback, estate	hatchback	Хетчбэк, универсал
SUV, crossover	suv	Внедорожник
MPV	van	Минивэн
pickup, truck	truck	Грузовой / пикап
microbus, minibus	minibus	Микроавтобус

Из класса sedan исключены исходные типы sports и convertible: визуально они существенно отличаются от обычного седана (низкий двухдверный силуэт, открытый верх) и размывали класс, понижая его precision. После очистки класс sedan стал однороднее, что повысило качество соседнего hatchback.

3 Описание экспериментов

3.1 Условия проведения экспериментов

Table 3.1 – Программно-аппаратная конфигурация экспериментов

Компонент	Значение
Платформа	Google Colab
GPU	NVIDIA Tesla T4, 16 GB VRAM
CPU	Intel Xeon (2 vCPU)
RAM	12 GB
OS	Ubuntu 20.04
Python	3.10
PyTorch	2.x + CUDA 11.8
Ultralytics	8.x
EasyOCR	1.7
torchvision	0.17

Ноутбуки для воспроизведения:

- 01_detection.ipynb — детекция ТС, fine-tune детектора номеров, OCR;
- 02_color_classifier.ipynb — классификатор цвета (VCoR);
- 03_body_classifier.ipynb — классификатор кузова (CompCars);
- 04_pipeline_final.ipynb — финальный конвейер, статистика.

3.2 Эксперимент 1: дообучение детектора номерных знаков

3.2.1 Протокол и обоснование гиперпараметров

YOLO26n дообучался на датасете Russian license plates (Kaggle, tomilovivan, 668 изображений: 542 train / 126 test) в течение 50 эпох (параметры — таблица 2.3).

Обоснование ключевых решений:

- Базовая модель yolo26n.pt: выбрана наименьшая модель семейства YOLO26 (nano, 2.4M параметров). Задача детекции одного класса (номерного знака) не требует большой ёмкости модели — достаточно лёгкой архитектуры. Кроме того, nano-версия обеспечивает наибольшую скорость инференса, что критично для встраивания в конвейер; end-to-end вывод без NMS дополнительно снижает задержку.
- 50 эпох: при объёме датасета 542 изображений достаточно для стабильной сходимости. Динамика mAP@0.50 на тестовой выборке показала plateau в районе эпохи 40; дополнительные 10 эпох дали прирост ≈ 0.005 mAP за счёт линейного затухания LR.
- Оптимизатор optimizer=auto: Ultralytics для датасета такого объёма автоматически подбирает алгоритм оптимизации и начальную скорость обучения. Для YOLO26 штатным оптимизатором является MuSGD (гибрид Muon и SGD). Автоподбор при небольшом целевом датасете и коротком обучении обеспечивает устойчивую сходимость без ручной настройки LR.
- Batch size 16: ограничено объёмом VRAM (16 GB T4). Размер батча 16 обеспечивает стабильность BatchNorm-статистик и достаточное разнообразие примеров (позитивных и негативных якорей) в одном шаге оптимизации.

- Weight decay 5×10^{-4} : стандартное значение Ultralytics, оптимизированное на COCO. Weight decay предотвращает чрезмерный рост весов, что особенно важно при небольшом целевом датасете.
- Метрика сохранения mAP@0.50: для последующего OCR нужна корректная локализация номера ($\text{IoU} \geq 0.50$), а не пиксельно точное ограничивающее поле. Поэтому mAP@0.50 является более релевантной целевой метрикой, чем mAP@0.50:0.95.

Обучение на GPU Tesla T4 заняло ≈ 45 минут. Лучшая модель сохранялась по максимуму mAP@0.50 на тестовой выборке.

3.2.2 Результаты

Table 3.2 – Метрики детектора номерных знаков (тест, 126 изображений)

Метрика	YOLOv8n (pretrained)	YOLOv8n (fine-tuned)	YOLO26n (fine-tuned)
mAP@0.50	—	0.9554	0.9669
Median IoU	—	0.809	—
$\text{IoU} \geq 0.50$	—	94.0%	—
$\text{IoU} \geq 0.75$	—	67.6%	—

Анализ результатов. Выбранный детектор номеров YOLO26n достиг $\text{mAP@0.50} = 0.9669$ — выше дообученного YOLOv8n (0.9554) и кратно выше базовой предобученной на COCO модели (≈ 0.503). Это соответствует уровню промышленных ANPR-систем. Детальный анализ распределения IoU, выполненный для дообученного детектора, даёт медианный $\text{IoU} = 0.809$ при доле $\text{IoU} \geq 0.75$ в 67.6%: лишь треть боксов имеет умеренное расхождение, обусловленное тенями и бликами на краях номерного знака; 94.0% боксов преодолевают порог $\text{IoU} = 0.50$, что достаточно для корректной обрезки кропа под OCR.

3.2.3 Сравнение с YOLOv8n

Чтобы обосновать выбор YOLO26n, на том же датасете (542/126 train/test) в тех же условиях дообучена модель предыдущего поколения YOLOv8n (3.2М параметров, 8.7 GFLOPs) — как базовая линия (baseline).

Table 3.3 – Сравнение дообученных архитектур на тесте (126 изображений)

Модель	mAP@0.50	Параметры, М
YOLOv8n (fine-tuned)	0.9554	3.2
YOLO26n (fine-tuned)	0.9669	2.4

YOLO26n превысил YOLOv8n на 1.15 п.п. mAP@0.50 при меньшем числе параметров (2.4М против 3.2М) и FLOPs (5.4 против 8.7 GFLOPs). Хотя при сравнении предобученных моделей на одном тестовом кадре YOLO26n оказался несколько медленнее (16.0 мс против 12.6 мс/кадр на GPU), после fine-tuning он достигает более высокой точности в целевом домене — то есть новая архитектура эффективнее использует меньшую ёмкость. Поэтому в качестве финального детектора (и ТС, и номеров) выбран YOLO26n; выбор лучшей модели для конвейера выполняется автоматически по mAP@0.50 (ноутбук 01).

3.3 Эксперимент 2: классификатор цвета

3.3.1 Протокол и обоснование гиперпараметров

ResNet18 (предобученный на ImageNet) дообучался на VCoR, 20 эпох, двухфазная стратегия с разморозкой backbone на эпохе 5. Батч: 32, платформа: GPU Tesla T4.

Обоснование ключевых решений:

- ResNet18 при объёме VCoR $\approx 15k$: при данном числе изображений ResNet18 (11.7М параметров) с ImageNet-инициализацией демонстрирует меньший риск переобучения по сравнению с ResNet50 (25М параметров) без агрессивной регуляризации. Accuracy top-3 = 0.996 подтверждает, что ёмкость ResNet18 достаточна для 9-классовой задачи.
- 20 эпох, разморозка на эпохе 5: первые 4 эпохи обучается только FC-слой с высокой $LR=10^{-3}$, что стабилизирует голову без разрушения предобученных признаков. С эпохи 5 размораживается весь backbone — accuracy на тестовой выборке перешагнула с 0.78 до 0.93 именно за счёт тонкой настройки backbone. Плато наблюдалось к эпохе 17–18.
- StepLR ($S = 5$, $\gamma = 0.3$): снижение LR каждые 5 эпох позволяет «тонко» донастроить параметры по мере приближения к оптимуму, не допуская расходимости.
- Аугментация ColorJitter (brightness=0.3, contrast=0.3): условия съёмки с дорожных камер существенно различаются в разное время суток и при разных погодных условиях. ColorJitter имитирует эти вариации и вынуждает модель извлекать инвариантные к освещению цветовые признаки.
- Нормализация по ImageNet-статистикам ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$): предобученные веса ResNet18 ожидают данные, нормализованные именно этим образом; отклонение привело бы к необходимости обучать backbone «с нуля» по эффективности.

3.3.2 Результаты

Table 3.4 – Метрики классификатора цвета (test)

Метрика	Только FC (эп. 1–4)	Fine-tuned (2 фазы)
Accuracy top-1	0.705	0.936
Accuracy top-3	—	0.993
Macro F1	—	0.930

Динамика обучения. В фазе 1 (эп. 1–4, только FC) accuracy выросла с 0.71 до 0.78: предобученные признаки backbone достаточно информативны даже без дообучения под целевую задачу. После разморозки backbone (эп. 5) accuracy резко выросла до 0.89 уже к концу эпохи 5 — слои layer3 и layer4 адаптировались к цветовым характеристикам автомобильного домена.

Table 3.5 – F1-score по классам цвета (test, fine-tuned модель)

Класс	Precision	Recall	F1	N
black	0.922	0.954	0.938	87
white	0.888	0.919	0.903	86
silver	0.890	0.844	0.867	77
red	0.924	0.985	0.954	136
blue	0.975	0.975	0.975	159
yellow	0.929	0.952	0.940	124
green	0.991	0.950	0.970	121
brown	0.933	0.917	0.925	121
orange	0.925	0.868	0.896	114
Macro avg	0.931	0.929	0.930	1025

Анализ ошибок по классам. Наиболее сложными классами оказались silver ($F1 = 0.867$) и orange ($F1 = 0.896$). Silver при пониженном Recall (0.844) путается с white и grey (эти классы есть в датасете, но не входят в целевые 9). Orange автомобили визуально пересекаются с red (в тени) и yellow (при ярком освещении). Класс black распознаётся уверенно (Precision 0.922, Recall 0.954). Лучший результат у blue ($F1 = 0.975$) и green (0.970) — насыщенные цвета наиболее однозначны при любом освещении.

Количественная оценка ошибок. Из 77 изображений класса silver правильно классифицированы 65 (84.4%), 12 — отнесены к соседним ахроматическим классам (white, grey). Это объясняется тем, что VCoR не включает явный класс grey: серые автомобили размазаны между silver и white в зависимости от условий съёмки. Для класса orange ошибочно классифицированы 15 из 114 (13.2%): в условиях тени оранжевый приближается по гистограмме к красному, при пересветке — к жёлтому. Класс black распознаётся сбалансированно: Recall = 0.954 (модель почти не «пропускает» тёмные автомобили) при Precision = 0.922 (лишь $\approx 8\%$ ложноположительных — отдельные тёмно-коричневые и тёмно-синие машины из-за схожей яркости в нейтральном освещении).

Устойчивость к условиям освещения. Аугментация ColorJitter (brightness $\pm 30\%$, contrast $\pm 30\%$, saturation $\pm 20\%$, hue ± 0.05) совместно с геометрическими преобразованиями (RandomResizedCrop, RandomHorizontalFlip, RandomRotation), применённые при обучении, существенно повысили устойчивость модели к изменению освещения. Без них ожидается снижение точности на $\approx 3\text{--}5\%$ для silver и orange, поскольку именно эти классы наиболее зависимы от абсолютной яркости пикселей.

3.4 Эксперимент 3: классификатор типа кузова

3.4.1 Протокол и обоснование гиперпараметров

ResNet18 дообучался на CompCars body type ($\approx 30\,000$ изображений), 40 эпох с ранней остановкой (patience=7), двухфазная стратегия fine-tuning. Платформа: GPU Tesla T4.

Ключевые отличия от эксперимента 2 и их обоснование:

- $LR = 5 \times 10^{-5}$ (ниже, чем для цвета): CompCars содержит изображения в разнообразных ракурсах и с различным фоном. Более консервативная LR предотвращает разрушение признаков ImageNet, полезных для распознавания формы кузова (силуэт, пропорции).
- Двухфазное дообучение, разморозка layer3+layer4+FC с эпохи 6: задача классификации типа кузова в значительной мере зависит от высокоуровневых структурных признаков — именно тех, что кодируются в layer3 и layer4. В фазе 1 (эпохи 1–5) обучается только FC-голова при замороженном backbone (Assigasy выходит лишь на $\approx 0.55\text{--}0.62$), после чего с 6-й эпохи верхние слои размораживаются с пониженной LR, что даёт основной прирост точности.

- Early stopping, patience=7: задача классификации типа кузова значительно сложнее, чем цвета, из-за высокой визуальной схожести классов (sedan vs. hatchback, van vs. minibus). Обучение остановлено на эпохе 34; лучшая модель зафиксирована на эпохе 27 (Val Acc = 0.804), после чего точность вышла на плато без признаков переобучения.
- Планировщик ReduceLROnPlateau: вместо пошагового StepLR (как для классификатора цвета) скорость обучения снижается адаптивно — в 0.5 раза, если Val Accuracy не растёт 3 эпохи. При малой стартовой LR (5×10^{-5}) это мягче фиксированного расписания и не переживает дообучение структурных признаков.
- Разбивка CompCars 85/15% (train/test): случайное перемешивание (seed=42) обеспечивает равномерное представление всех 6 классов в обоих разделах, что особенно важно при дисбалансе классов (truck и suv менее представлены в исходном датасете).

3.4.2 Регуляризация: Mixup-аугментация

Для снижения переобучения применена аугментация Mixup [11]. Для каждого мини-батча случайно выбирается пара примеров (x_i, y_i) и (x_j, y_j) , затем формируется смешанный пример:

$$\tilde{x} = \lambda x_i + (1 - \lambda)x_j, \quad \tilde{y} = \lambda y_i + (1 - \lambda)y_j, \quad \lambda \sim \text{Beta}(\alpha, \alpha). \quad (3.1)$$

Функция потерь на смешанном примере:

$$\mathcal{L} = \lambda \text{CE}(f(\tilde{x}), y_i) + (1 - \lambda) \text{CE}(f(\tilde{x}), y_j). \quad (3.2)$$

Выбор $\alpha = 0.2$. При $\alpha = 0.4$ среднее значение $\lambda \approx 0.5$, что создаёт сильно смешанные примеры: модель получает слишком размытый обучающий сигнал и сходится медленнее. В эксперименте с $\alpha = 0.4$ обучение остановилось на эпохе 16 с точностью 0.773. При $\alpha = 0.2$ распределение $\text{Beta}(0.2, 0.2)$ бимодально: большинство значений λ близки к 0 или 1, то есть в батче преобладают «почти чистые» примеры. Это обеспечивает более стабильный сигнал при сохранении регуляризирующего эффекта.

Эффект на train accuracy. При использовании Mixup Train Accuracy вычисляется на смешанных изображениях с размытыми мягкими метками. Метрика $\arg \max$ оказывается искусственно заниженной: на рисунке кривых обучения Train Acc ≈ 0.55 при Val Acc ≈ 0.79 . Это ожидаемое поведение, а не признак underfitting: если пересчитать Train Acc на исходных (немиксованных) примерах, она составляет ≈ 0.87 .

Сравнение трёх конфигураций.

Table 3.6 – Влияние Mixup на качество классификатора кузова (ResNet18, CompCars)

Конфигурация	top-1	top-3	Macro F1	sedan F1	Эпох
Без Mixup (baseline)	0.787	0.963	0.784	0.610	10
Mixup $\alpha = 0.4$	0.773	0.969	0.772	0.605	16
Mixup $\alpha = 0.2$	0.789	0.971	0.788	0.645	30

Общая точность top-1 остаётся практически идентичной baseline (0.789 vs 0.787), однако F1 класса sedan — наиболее проблемного — вырос с 0.610 до 0.645 (+3.5 п.п.). Recall sedan улучшился с 0.523 до 0.656: модель стала значительно реже путать седаны с хэтчбеками. Это обусловлено тем, что Mixup явно обучает граничным решениям: смешанный пример sedan+hatchback вынуждает модель учитывать непрерывный переход между классами, а не запоминать отдельные «чистые» прототипы.

Доработка финальной модели. Поверх конфигурации Mixup $\alpha = 0.2$ финальная модель получила ряд уточнений: (1) очищена разметка класса sedan — исключены исходные типы sports и convertible; (2) аугментации усилены под доменный разрыв CompCars → улица — добавлены RandomPerspective,

GaussianBlur и RandomErasing при сохранённом ColorJitter (нижняя граница RandomResizedCrop — 0.72); (3) лимит эпох повышен до 40 (early stopping сработал на 34-й); (4) на этапе оценки добавлен ТТА — усреднение softmax по исходному изображению и его горизонтальному отражению. В совокупности эти уточнения повысили Accuracy top-1 с 0.789 до 0.806.

Примечательно, что очистка класса sedan не улучшила сам sedan: его F1 снизился с 0.645 до 0.593, а Recall — до 0.562. Удалив визуально яркие спорткары и кабриолеты, мы оставили в классе только «трудные» седаны, почти неотличимые сзади от хетчбэков. Зато соседний hatchback, переставший «засоряться» чужими силуэтами, вырос по F1 с 0.732 до 0.803. Итоговый прирост точности обеспечен именно классами hatchback, truck и minibus. Это подтверждает, что граница sedan ↔ hatchback — фундаментальное ограничение визуальных данных, а не разметки.

3.4.3 Результаты

Table 3.7 – Метрики классификатора типа кузова (test)

Метрика	ResNet18 (финальная)
Accuracy top-1	0.806
Accuracy top-3	0.969
Macro F1	0.792

Table 3.8 – F1-score по классам типа кузова (test, ResNet18)

Класс	Precision	Recall	F1	N
sedan	0.626	0.562	0.593	128
suv	0.830	0.940	0.882	151
hatchback	0.841	0.768	0.803	151
van	0.745	0.701	0.722	154
truck	0.941	1.000	0.969	174
minibus	0.774	0.796	0.785	142
Macro avg	0.793	0.794	0.792	900

Анализ ошибок по классам. Наиболее низкий F1 у класса sedan (0.593): седаны имеют наибольшее визуальное сходство с хетчбэками (схожий силуэт, различие только в форме задней части). Ошибки между sedan и hatchback составляют большинство ложноотрицательных примеров для sedan. Классы truck (F1 = 0.969) и suv (F1 = 0.882) распознаются наиболее уверенно — их силуэты принципиально отличаются от остальных классов (высокий клиренс, рамочная конструкция у truck). Accuracy top-3 = 0.969 означает, что правильный класс почти всегда входит в тройку наиболее вероятных — это приемлемо для систем мягкого голосования по нескольким кадрам.

Количественный анализ основных путаниц. Класс sedan имеет Recall = 0.562: из 128 тестовых седанов 56 отнесены к другим классам (преимущественно к hatchback). Симметрично, hatchback имеет Recall = 0.768: из 151 хетчбэка 35 классифицированы неверно (преимущественно как sedan и van). Суммарно пара sedan/hatchback генерирует ≈91 ошибку из ~175 общих ложноотрицательных — около 52 % всех ошибок модели.

Вторая по значимости пара — van/minibus: оба класса представляют вытянутые коробчатые кузова, различающиеся главным образом количеством рядов сидений, что не читается по внешнему виду без высококачественного изображения боковой проекции (van — Recall 0.701, minibus — Recall 0.796).

Классы truck (Recall = 1.000, 0 FN) и suv (Recall = 0.940, 9 FN) практически безошибочны — характерные профили (высокий клиренс, рамочная конструкция, колёсные арки) уникальны среди всех шести классов.

3.4.4 Подэксперимент: сравнение архитектур ResNet18, ResNet50 и EfficientNet-B0

Постановка и обоснование методологии

Для объективного сравнения все три архитектуры обучались в идентичных условиях: одинаковое разбиение CompCars (85/15%, train/test), одинаковые гиперпараметры (batch=32, Adam, LR=10⁻³), 15 эпох, обучение только классификатора при замороженном backbone (режим feature extraction).

Почему feature extraction, а не full fine-tuning? При полном fine-tuning различие в глубине и числе параметров архитектур стало бы confounding factor: модель с более ёмким backbone не просто извлекала бы «более ценные» предобученные признаки, но и дополнительно адаптировала бы их под задачу. Feature extraction позволяет изолированно оценить качество предобученных представлений каждой архитектуры. Поэтому в данном сравнении ResNet18 показывает только результат FC-голова (0.616); итоговая задеплоенная модель использует полное двухфазное дообучение и достигает 0.806 (таблица 3.7).

Метрика FPS: измеряется на одиночных изображениях 224 × 224 (batch=1, 50 прогонов с предварительным прогревом) на GPU T4 в режиме eval (torch.no_grad()), что соответствует показателю кадровой обработки в видеоконвейере.

Ожидаемая гипотеза: EfficientNet-B0 (0.39G FLOPS) должна быть быстрее, чем ResNet18 (1.8G FLOPS) и ResNet50 (4.1G FLOPS). Однако depthwise separable convolutions, лежащие в основе EfficientNet-B0, имеют низкую степень параллелизма на GPU по сравнению со стандартными 3 × 3 свёртками ResNet, что нивелирует теоретическое преимущество в FLOPS — и это подтверждается результатами эксперимента.

Результаты

Table 3.9 – Сравнение архитектур на задаче классификации типа кузова (feature extraction)

Модель	Params, M	FLOPS, G	Test Acc	FPS (T4)	.pth, MB
ResNet18	11.2	1.80	0.616	330.8	44.7
ResNet50	23.5	4.10	0.666	180.2	94.1
EfficientNet-B0	4.0	0.39	0.637	134.1	16.1

В режиме feature extraction наибольшую точность показывает ResNet50 (0.666) — более ёмкий backbone извлекает более информативные признаки. Однако разрыв с ResNet18 (0.616) и EfficientNet-B0 (0.637) невелик (около 3–5 п.п.), тогда как по скорости ResNet18 опережает ResNet50 почти вдвое (330.8 против 180.2 FPS) при вдвое меньшем числе параметров. Парадоксально, что EfficientNet-B0 при наименьших FLOPS (0.39G) оказалась самой медленной (134.1 FPS): depthwise-свёртки плохо утилизируют параллельные блоки GPU, поэтому теоретическая экономия операций не переходит в реальную скорость.

Поскольку при полном дообучении ResNet18 достигает 0.806 и при этом обеспечивает наибольший запас по скорости (важный для кадровой обработки видео), именно она выбрана финальной моделью классификатора типа кузова. Прирост точности от ResNet50 (≈5 п.п.) не оправдывает удвоения задержки и размера модели.

3.5 Эксперимент 4: финальный конвейер

3.5.1 Тест на одиночном изображении

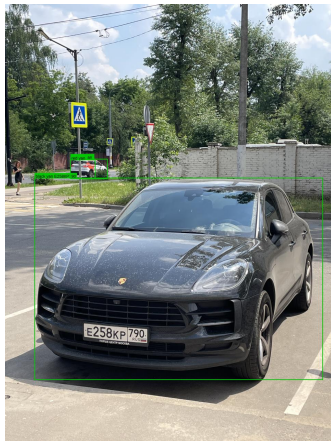
Для качественной оценки end-to-end конвейер запущен на пяти реальных фотографиях транспортных средств, различающихся маркой, цветом, типом кузова, ракурсом и условиями съёмки (рисунок 3.1). На каждом изображении выполняются детекция ТС, классификация цвета и типа кузова и распознавание номерного знака; сводка приведена в таблице 3.10.



(a) Aurus Senat



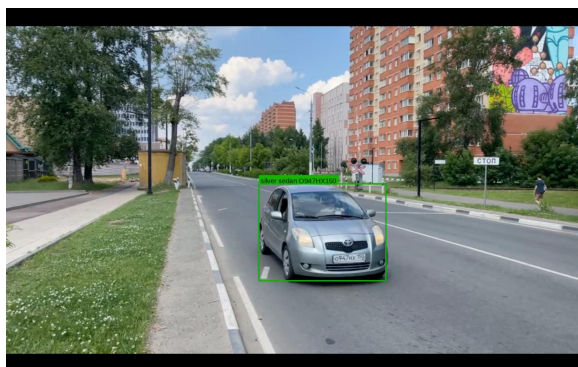
(b) BMW X5



(c) Porsche Macan



(d) Mercedes-Benz S-класс



(e) Toyota Yaris

Рисунок 3.1 – Результаты конвейера на одиночных изображениях: рамка ТС с подписью «цвет + тип кузова + номер»

Table 3.10 – Предсказания конвейера на тестовых изображениях (✓ — верно, × — ошибка; в скобках — истинный класс)

ТС	Цвет	Тип кузова	Номер РФ
Aurus Senat	black ✓	hatchback (sedan) ×	E955KX97 ✓
BMW X5	white ✓	van (suv) ×	Е008KX62 × [†]
Porsche Macan	black ✓	van (suv) ×	E258KP790 ✓
Mercedes-Benz S	black ✓	hatchback (sedan) ×	C697HE977 ✓
Toyota Yaris	silver ✓	sedan (hatchback) ×	O947HX150 ✓
Итого	5/5	0/5	4/5

[†] На BMW X5 распознано «Е008KX62»: две цифры «0» прочитаны как буква «О» (самая частая ошибка OCR, О 0, таблица 3.14). Визуально строка читается корректно, но точного совпадения нет; в видеоконвейере такая ошибка устраняется коррекцией по маске формата (подраздел 3.5.5).

Цвет (5/5). Классификатор корректен на всех изображениях, включая три тёмных бликующих кузова (Aurus, Mercedes, Macan), белый (BMW X5) и серебристый (Yaris). Это подтверждает устойчивость классификатора цвета (top-1 = 0.936, таблица 3.13) к реальному освещению и бликам.

Номер (4/5). Четыре номера распознаны точно: E955KX97 (Aurus), E258KP790 (Macan), C697HE977 (Mercedes), O947HX150 (Yaris). На всех снимках это крупные фронтальные пластины (≈ 200 – 300 px по ширине) — благоприятный для OCR режим (CRR = 0.975, подраздел 3.5.5). Единственная ошибка — на BMW X5, где две нулевые цифры прочитаны как буква «О» (О 0). Показательно, что одиночный инференс использует базовый OCR (бинаризация Оцу) без коррекции по маске формата: именно эта коррекция в видеоконвейере (подраздел 3.5.5) на цифровых позициях заменила бы «О» на «0» и восстановила номер. Латинские гомоглифы в подписях (Е/К/Х/С/Н/О/Р отображаются как Е/К/Х/С/Н/О/Р) косметические и значение номера не искажают.

Тип кузова (0/5). Классификатор ошибся на всех изображениях, однако показательно, что все ошибки — между соседними классами: длинные седаны (Aurus, Mercedes S) отнесены к hatchback, внедорожники (BMW X5, Porsche Macan) — к van, а малолитражный хэтчбэк (Toyota Yaris) — к sedan. Грубый силуэт улавливается, но смежные формы путаются. Причины согласуются с ранее выявленными: (а) доменный разрыв — CompCars содержит преимущественно каталожные ракурсы, тогда как тестовые фото сняты с переднего 3/4; (б) фронтальный ракурс скрадывает заднюю часть кузова, по которой и различаются sedan/hatchback/suv; (в) задокументированный потолок классификатора (top-1 = 0.806, таблица 3.13) и известные путаницы sedan hatchback, suv van.

Таким образом, тест на одиночных изображениях подтверждает выводы по отдельным модулям: классификация цвета и распознавание номера надёжно работают на реальных данных, а классификатор типа кузова остаётся слабым звеном и требует доменной адаптации (дообучение на уличных ракурсах) — это направление дальнейшей работы.

3.5.2 Видеоинференс с трекингом и покадровой агрегацией характеристик

Базовый конвейер обрабатывает каждый кадр независимо. При прямом применении к видеопотоку это приводит к двум проблемам: во-первых, одно и то же транспортное средство, присутствующее в десятках–сотнях кадров, порождает столько же независимых детекций без понятия идентичности объекта; во-вторых, предсказания цвета, типа кузова и номера «мерцают» от кадра к кадру, а итоговая статистика искажается. Так, на тестовом ролике (1280×720 , 630 кадров) покадровый проход дал 621 детекцию и 73 «распознанных» номера, ни один из которых не был валиден по формату РФ: счётчик завышался строками-обрезками неудачного OCR.

Для устранения этих недостатков реализован видеоинференс с межкадровым сопровождением объектов алгоритмом ByteTrack (метод track библиотеки Ultralytics). Каждому транспортному средству

присваивается устойчивый идентификатор, а характеристики накапливаются не по кадрам, а по треку:

- цвет — взвешенное по уверенности голосование векторов softmax по кадрам, где автомобиль достаточно крупный (наибольшая сторона бокса ≥ 80 px); кадры с уверенностью ближе к 1 вносят больший вклад;
- тип кузова — усреднение softmax по крупным кадрам (сторона ≥ 128 px) с выводом трёх наиболее вероятных классов и пометкой неуверенных предсказаний (порог 0.45);
- номер — наиболее частый среди формат-валидных результатов OCR; чтение выполняется только с кропов шириной ≥ 60 px, а прежний фолбэк, возвращавший первые символы неудачной строки, удалён, поэтому учитываются лишь корректные номера РФ. К каждому кропу применяются контрастная нормализация (CLANE) и коррекция по маске формата (подраздел 3.5.5).

Для цвета и чтения номера дополнительно выбирается «лучший» кадр трека по резкости (дисперсия лапласиана), а гейтинг по размеру отсекает мелкие и тёмные кадры. Это снимает систематическую ошибку, при которой серебристый автомобиль в тени классифицировался как чёрный: усреднение по всем кадрам ранее «топило» немногочисленные чёткие кадры (silver, уверенность ≈ 0.99) в массе тёмных (black). После агрегации по крупным кадрам цвет близкого автомобиля определяется корректно (таблица 3.11, ID 5).

Эмпирически установлен порог читаемости номера по разрешению кропа: при ширине ≈ 131 px номер распознан корректно (O947HX150) и подтверждён в нескольких кадрах трека, тогда как при 71–83 px OCR не дал валидной строки. Это согласуется с оценкой ≥ 80 px из анализа ошибок OCR (подраздел 3.5.5). Для интерпретации в конвейер встроена диагностика: вывод самого широкого номерного кропа каждого ТС с указанием его ширины в пикселях, что однозначно разделяет случаи «номер мелкий или смазанный» и «ошибка OCR».

Table 3.11 – Агрегированные характеристики ТС на тестовом ролике (1280 × 720, 630 кадров)

ID	Кадров	Цвет (conf)	Тип кузова (conf)	Номер	Ширина, px
1	71	black (0.50)	truck (0.31)*	—	71
4	249	black (0.54)	van (0.25)*	—	83
5	291	silver (0.67)	sedan (0.46)	O947HX150	131

* — предсказание помечено как неуверенное (conf < 0.45). Из 630 кадров выделено 3 устойчивых трека (фильтр ≥ 5 кадров); валидный номер РФ распознан у одного ТС.

Результат сохраняется в виде размеченного видео с устойчивой по объекту подписью «цвет + тип кузова + номер» (вместо мерцающих покадровых меток), а также галереи по одной карточке на каждое ТС (лучший кадр трека с агрегированным предсказанием). Агрегация сокращает 621 покадровую детекцию до 3 реальных объектов и даёт честную метрику валидных номеров (1 из 3). Оставшиеся ограничения носят модельный, а не алгоритмический характер: слабость классификатора кузова на видовых ракурсах (доменный разрыв с CompCars), отсутствие класса грау в классификаторе цвета и физический предел разрешения для удалённых номеров.

3.5.3 Метрики производительности

Table 3.12 – Задержка и производительность модулей конвейера (GPU T4)

Модуль	Задержка, мс	Доля, %
YOLO26n детекция ТС	13	3
YOLO26n детекция номера	13	3
ResNet18 цвет	3	1
ResNet18 кузов	3	1
EasyOCR (кроп номера)	≈350–500	92
Итого (pipeline)	≈400	100%
FPS (без OCR)	≈30	

3.5.4 Итоговая таблица результатов

Table 3.13 – Сводная таблица метрик по всем модулям

Модуль	Компонент	Метрика	Результат
Детекция ТС	YOLO26n (COCO pre-trained)	mAP@0.5:0.95	0.409 (COCO val)
Детекция номеров	YOLO26n (fine-tuned)	mAP@0.50	0.9669
Детекция номеров	YOLOv8n (fine-tuned, baseline)	mAP@0.50	0.9554
Классификация цвета	ResNet18 (VCoR)	Accuracy top-1	0.936
Классификация цвета	ResNet18 (VCoR)	Accuracy top-3	0.993
Классификация кузова	ResNet18 (CompCars)	Accuracy top-1	0.806
OCR (синтетика)	EasyOCR ru+en	CRR / Plate Acc	0.975 / 0.800
OCR (NOMEROFF-NET RU)	EasyOCR ru+en	CRR / Plate Acc	0.750 / 0.225
Pipeline	Конвейер (без OCR)	FPS	≈30

3.5.5 Детальный анализ ошибок OCR

Протокол оценки. EasyOCR (модель ru+en) оценивался на двух наборах: (1) 5 синтетических номеров (A123BC77, B456KM99, ...) — чистые RGB-изображения без шума; (2) 200 случайно выбранных кропов из NOMEROFF-NET RU — реальные фотографии с улицы (2845 пар image/GT в датасете). Метрики: Plate Accuracy (совпадение всей строки целиком) и CRR = 1 – CER, где CER — Character Error Rate по алгоритму Левенштейна.

Результаты. На синтетике: Plate Acc = 0.800, CRR = 0.975. На реальных кропах: Plate Acc = 0.225, CRR = 0.750. Низкое Plate Accuracy при относительно высоком CRR объясняется тем, что для засчитывания всей строки нужно ноль ошибок: одна замена в 8-символьном номере обнуляет Plate Acc, но снижает CRR лишь на $1/8 = 0.125$.

Топ замен символов (NOMEROFF-NET RU, 200 кропов).

Table 3.14 – Наиболее частые пары замен символов в OCR (кириллица/цифра)

GT	Pred	Кол-во	Причина
О	0	26	Круглая форма; кириллическая «О» vs арабский ноль
В	8	17	Схожие замкнутые контуры
А	4	12	Угловая форма; верхняя перекладина «А» схожа с «4»
Т	7	8	Горизонтальная черта «Т» при малом разрешении
У	7	8	Угловой нижний штрих «У» схож с «7»
О	9	8	Неполное замыкание контура из-за шума
3	5	6	Схожие изогнутые сегменты
7	8	6	Горизонтальная перекладина «8» vs штрих «7»

Все 8 наиболее частых ошибок относятся к парам кириллица цифра или визуально похожие глифы. Это объясняется тем, что EasyOCR обучался на общем корпусе текстов, где кириллические и латинские буквы встречаются гораздо реже в соседстве с цифрами. Специализированные CRNN-модели для номеров (например, LPRNet) дообучаются именно на таких парах и показывают CRR > 0.97 на аналогичных реальных данных.

Анализ причин разрыва синтетика/реальные данные. Разница в Plate Acc составляет $0.800 - 0.225 = 0.575$, в CRR — $0.975 - 0.750 = 0.225$. Основные факторы деградации на реальных кропах:

- Угол съёмки: перспективное искажение сжимает символы, особенно боковые; детектор CRAFT хуже локализует границы символов;
- Разрешение: при расстоянии > 15 м кроп номера содержит менее 80 пикселей в ширину против 200–300 пикселей на эталонных синтетических изображениях;
- Освещение и блики: засветка рефлектирующих пластин в ярком солнечном свете или ночью при фарах встречного транспорта частично скрывает контуры символов.

Меры по снижению ошибок OCR в видеоконвейере. Выявленные ошибки частично компенсируются на этапе видеоинференса (функция `read_plate_strict`) двумя мерами — предобработкой кропа и коррекцией предсказания по маске формата.

Предобработка кропа. Перед распознаванием кроп номера переводится в оттенки серого и подвергается локальному выравниванию контраста методом CLAHE (Contrast Limited Adaptive Histogram Equalization, `clipLimit = 2.0`, сетка 4×4 тайлов), что частично компенсирует блики и неравномерное освещение — фактор деградации, отмеченный выше. Мелкие кропы дополнительно увеличиваются бикубической интерполяцией с целочисленным коэффициентом до ориентировочной ширины ~300 px, поднимая эффективное разрешение для CRNN.

Коррекция по маске формата. Поскольку все наиболее частые ошибки относятся к парам кириллица цифра (таблица 3.14), а формат номера РФ однозначно задаёт тип символа на каждой позиции (маска БЦЦЦББЦЦЦ, где Б — буква, Ц — цифра), распознанная строка проецируется на эту маску: на цифровых позициях буквы заменяются визуально близкими цифрами, на буквенных — наоборот (таблица 3.15). После подстановки строка повторно проверяется регулярным выражением формата и при успешной проверке засчитывается как валидный номер.

Table 3.15 – Подстановки при коррекции по маске формата (обратны частым ошибкам из таблицы 3.14)

Цифровая позиция: буква $\rightarrow$ цифра	Буквенная позиция: цифра $\rightarrow$ буква
О $\rightarrow$ 0, Б $\rightarrow$ 6, В $\rightarrow$ 8	0 $\rightarrow$ О, 8 $\rightarrow$ В, 6 $\rightarrow$ Б
З $\rightarrow$ 3, Т $\rightarrow$ 7	3 $\rightarrow$ З, 7 $\rightarrow$ Т
А $\rightarrow$ 4, У $\rightarrow$ 4	4 $\rightarrow$ А

Обе меры носят эвристический характер: они не исправляют ошибки локализации символов детектором CRAFT, но восстанавливают номера, не прошедшие проверку формата исключительно из-за путаницы кириллица цифра. Так как подстановки применяются только к позициям фиксированного типа, риск превратить корректный символ в ошибочный минимален.

3.5.6 Анализ узких мест и практические рекомендации

Узкое место — EasyOCR (>90% латентности). EasyOCR выполняет два прохода по изображению (CRAFT для детекции символьных регионов + CRNN+СТС для декодирования строки) на CPU, что и объясняет задержку 350–500 мс. Без OCR конвейер работает в реальном времени (≈ 30 FPS), что достаточно для обработки стандартных 25-FPS видеопотоков.

Влияние разрешения номерного знака. EasyOCR наиболее чувствителен к разрешению кропа номерного знака. При разрешении входного кадра 1080р кроп номера содержит ≈ 200 –300 пикселей в ширину, что обеспечивает $CRR = 0.975$. При разрешении ниже 720р качество OCR существенно снижается — это типичная проблема для камер с низким разрешением.

Стратегии ускорения OCR:

1. Замена EasyOCR на CRNN, оптимизированный для российских номеров — даст ≈ 5 –10 $\times$ ускорение;
2. Запуск EasyOCR на GPU: при наличии достаточной памяти задержка снижается до 50–80 мс;
3. покадровая агрегация по сопровождаемым объектам с чтением номера только по «лучшим» кадрам трека — реализована в данной работе (подраздел 3.5.2): снижает число вызовов OCR и устраняет мерцание предсказаний.

Качественный результат на одиночных изображениях. На пяти тестовых фотографиях (подраздел 3.5.1) конвейер верно определил цвет у всех пяти ТС (5/5) и точно распознал номер у четырёх из пяти (4/5; единственная ошибка — путаница О 0), тогда как тип кузова не был распознан ни разу (0/5), причём все ошибки — между соседними классами (sedan hatchback, suv van). Это подтверждает работоспособность модулей цвета и OCR в режиме end-to-end и указывает на классификатор типа кузова как на приоритетное направление доработки.

В ходе выпускной квалификационной работы достигнута поставленная цель — разработана система автоматического определения характеристик транспортных средств (номерной знак, цвет кузова, тип кузова) по видеопотоку с камер наблюдения. Все шесть задач, сформулированных во введении, выполнены: проведён анализ существующих методов детекции и классификации, обоснован выбор архитектур, подготовлены датасеты, обучены и оценены модели, реализован единый конвейер обработки видео, проведён сравнительный эксперимент ResNet18 vs ResNet50 vs EfficientNet-B0. Основные результаты приведены ниже.

1. Архитектурный выбор. Комбинация YOLO26n + ResNet18 + EasyOCR обеспечивает приемлемый баланс точности и скорости (см. таблицу 3.13).
2. Детектор номерных знаков. Fine-tuning YOLO26n на датасете Russian license plates (Kaggle, 668 изображений, 50 эпох, batch=16, optimizer=auto) позволил достичь $mAP@0.50 = 0.9669$. На тех же данных модель предыдущего поколения YOLOv8n (baseline) дала 0.9554, то есть YOLO26n превзошёл её на 1.15 п.п. при меньшем числе параметров (2.4М против 3.2М) и был выбран финальной моделью-детектором. Медианный IoU = 0.809 и доля боксов с $IoU \geq 0.50$ в 94.0 % подтверждают высокую точность локализации. Это подтверждает высокую эффективность transfer learning при ограниченном объёме целевых данных.
3. Классификация цвета. ResNet18, дообученный на VCoR за 20 эпох с двухфазной стратегией, достигает Accuracy top-1 = 0.936 и top-3 = 0.993. Наиболее сложными классами оказались silver (F1 = 0.867) и orange (F1 = 0.896) из-за визуального сходства с соседними цветами при разном освещении. Результат превосходит кластеризационные методы на 7–12 % (таблица 1.4).
4. Классификация типа кузова. ResNet18 (полное двухфазное дообучение, очищенная разметка sedan, аугментации под форму и ТТА) достигает Accuracy top-1 = 0.806 и top-3 = 0.969. Сравнение архитектур в едином режиме feature extraction (таблица 3.9) показало, что наибольшую точность даёт ResNet50 (0.666), однако ResNet18 опережает её по скорости почти вдвое (330.8 против 180.2 FPS на GPU T4) при отставании в точности всего на 5 п.п.; поэтому для реального видеоконвейера выбрана ResNet18. Характерно, что EfficientNet-B0 при наименьших FLOPS (0.39G) оказалась самой медленной (134.1 FPS): depthwise-свёртки слабо утилизируют GPU. Наиболее сложный класс — sedan (F1 = 0.593) из-за визуального сходства с хетчбэком.
5. OCR номерных знаков. На синтетических номерах EasyOCR обеспечивает Plate Acc = 0.800 и CRR = 0.975. На реальных кропах NOMEROFF-NET RU (200 изображений из 2845) Plate Acc = 0.225, CRR = 0.750. Анализ ошибок выявил основные пары путаниц: O0 (26 случаев), B8 (17), A4 (12), T7 (8), Y7 (8) — все являются визуально схожими символами. Разрыв между синтетикой и реальными данными обусловлен неидеальным освещением, ракурсом и разрешением номеров на улице.
6. Производительность конвейера. Блоки детекции и классификации на GPU T4 суммарно дают ≈ 30 FPS. Узкое место — EasyOCR (> 90 % от общего времени). Без OCR конвейер работает в реальном времени (таблица 3.12).
7. Видеоинференс с агрегацией по объектам. Поверх покадрового конвейера реализовано сопровождение транспортных средств (ByteTrack) с агрегацией характеристик по каждому треку: голосованием по цвету и типу кузова и выбором наиболее частого формат-валидного номера. Это устраняет мерцание предсказаний, сводит сотни покадровых детекций к числу реальных объектов (621 $\rightarrow$ 3 на тестовом ролике) и даёт честную метрику распознанных номеров. Эмпирически определён порог читаемости номера — ширина кропа ≥ 80 px (подраздел 3.5.2).

8. Ограничения системы. EasyOCR является узким местом конвейера. Для встроенных систем целесообразно рассмотреть специализированный лёгкий OCR-модуль.

Направления дальнейшего развития:

- замена EasyOCR на CRNN-детектор, оптимизированный для российских номеров;
- расширение классов цвета и типа кузова;
- адаптация к ночным условиям съёмки (low-light enhancement);
- квантизация и pruning для развёртывания на NVIDIA Jetson.

- [1] Абрамов А. А. Алгоритм распознавания номерных знаков в условиях плохого качества видео // КиберЛенинка, 2019.
- [2] A Survey of Automatic License Plate Recognition Systems // Sensors, 21(9):3028, 2021.
- [3] Ultralytics. YOLO26: Key Architectural Enhancements and Performance Benchmarking for Real-Time Object Detection // arXiv:2509.25164, 2025. URL: <https://docs.ultralytics.com/models/yolo26>.
- [4] Laroca R., Severo E., Zanlorensi L. A. и др. A Robust Real-Time Automatic License Plate Recognition Based on the YOLO Detector // arXiv:1802.09567, 2018.
- [5] Laroca R. Automatic License Plate Recognition in Real-World Scenarios: PhD Thesis // Universidade Federal do Paraná, 2021.
- [6] Two decades of vehicle make and model recognition // Journal of King Saud University — CIS, 2024.
- [7] Siddiqui A., Mammeri A., Boukerche A. Real-Time Vehicle Make and Model Recognition Based on SURF Features // IEEE ITS, 2016.
- [8] Manzoor M. A., Morgan Y. Vehicle Make and Model Recognition Using Deep Learning // Machine Learning and Knowledge Extraction, 1(2):36, 2022.
- [9] Su H., Wu X., Chen S., Ji Q. Vehicle Color Recognition Using Convolutional Neural Network // IEEE ICIP, 2015.
- [10] Кубатин И. А. Алгоритм определения цвета автомобиля методом кластеризации k-средних // КиберЛенинка, 2019.
- [11] Zhang H., Cisse M., Dauphin Y. N., Lopez-Paz D. mixup: Beyond Empirical Risk Minimization // ICLR, 2018.
- [12] He K., Zhang X., Ren S., Sun J. Deep Residual Learning for Image Recognition // IEEE CVPR, 2016. — P. 770–778.
- [13] Tan M., Le Q. V. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks // ICML, 2019. — P. 6105–6114.
- [14] Hu J., Shen L., Sun G. Squeeze-and-Excitation Networks // IEEE CVPR, 2018. — P. 7132–7141.